SELENIUM TESTING GUIDE

Software Defect Prediction System

Automation testing of the web application with Selenium WebDriver, Python and pytest. Every step from an empty laptop to a finished, exam-ready test submission, written for complete beginners. No step is skipped.

What you need	A Windows 10 or 11 laptop (64-bit), internet, about 3 GB of free space, permission to install programs, and Google Chrome
Time	About 45-60 minutes for the first setup (mostly waiting for downloads). After that, about 1 minute to start everything
What you will have	70 working Selenium tests of the app (login, registration, validation, file upload, reports, dropdowns, roles, alerts, windows, waits), an HTML report with a screenshot of every test, and a recipe for writing new tests in the exam
Exam focus	Positive / negative / empty test cases, data-driven tests, reusable steps (Page Objects), assertions, evidence, documenting failures, viva answers

The parts of this guide

1. **Part A, Set up**: install the tools and run the app (Steps 1-12)
2. **Part B, Know the app**: roles, pages and expected messages (Steps 13-15)
3. **Part C, Selenium basics**: how it works, locators, your first script (Steps 16-18)
4. **Part D, Run the tests**: all tests, watching the browser, HTML report, evidence (Steps 19-24)
5. **Part E, How the framework is built**: fixtures, Page Objects, waits (Steps 25-28)
6. **Part F, Exam scenarios**: six complete, explained test files (Steps 29-34)
7. **Part G, Write a new test yourself**: the 8-step recipe and a worked example (Steps 35-37)
8. **Part H, Results and submission** (Steps 38-41) **Part I, Exam day** (Steps 42-43)
9. **Part J, Viva questions** **Part K, Troubleshooting** **Appendices**: command card, locator list, Selenium cheat sheet

Project: https://github.com/Debdeeptax/defect-predictor · Guide version: 5 November 2026 · Every command and every test in this guide was run and passed on Windows 11, Python 3.11, Google Chrome 157, Selenium 4.52, pytest 8.4.

Contents

Before you start: how to use this guide . . . 4

Part A: Set up the tools and the application . . . 6

Step 1: Open PowerShell . . . 6

Step 2: Install Python 3.11 . . . 6

Step 3: Install Google Chrome and check its version . . . 7

Step 4: (Optional) Install Git . . . 7

Step 5: Download the project . . . 7

Step 6: Go into the project folder and check the test files . . . 8

Step 7: Create the project's private Python folder (virtual environment) . . . 8

Step 8: Install the project's packages . . . 9

Step 9: Create the settings file (.env) . . . 9

Step 10: Create the database with demo users and a sample report . . . 10

Step 11: Start the app and log in by hand . . . 10

Step 12: Stop the app (and start it again on another day) . . . 11

Part B: Know the application you are testing . . . 12

Step 13: Learn the three roles . . . 12

Step 14: Take the guided tour . . . 12

Step 15: Write down the exact messages (your expected results) . . . 16

Part C: Selenium basics in 15 minutes . . . 18

Step 16: Understand how a Selenium test works . . . 18

Step 17: Find elements: locators and the data-testid attribute . . . 18

Step 18: Write and run your first Selenium script . . . 19

Part D: Run the project's automated tests . . . 21

Step 19: Run the whole Selenium test suite . . . 21

Step 20: Watch the browser while the tests run . . . 21

Step 21: Run only part of the tests . . . 22

Step 22: Create the HTML test report . . . 22

Step 23: Find the screenshot evidence . . . 23

Step 24: (Optional) Two extra ways to run tests . . . 23

Part E: How the test framework is built . . . 24

Step 25: The folder map . . . 24

Step 26: Fixtures: what happens before and after each test . . . 24

Step 27: Page Object Model: the reusable steps . . . 25

Step 28: Waits and sync markers: never use sleep . . . 27

Part F: Exam scenarios: six complete test files . . . 28

Step 29: Scenario 1: Login (data-driven, positive, negative, empty) . . . 28

Step 30: Scenario 2: Registration and form validation . . . 30
Step 31: Scenario 3: File upload and prediction . . . 31
Step 32: Scenario 4: Report page, sorting, dynamic content, new tab . . . 33
Step 33: Scenario 5: Dropdowns (filter, compare, admin role) and user management . . . 35
Step 34: Scenario 6: Alerts, navigation, cookies, waits, scrolling, access control . . . 36
Part G: Write a new test yourself (what you do in the exam) . . . 39
Step 35: The 8-step recipe for any scenario . . . 39
Step 36: Worked example from scratch . . . 39
Step 37: A blank template to start from . . . 40
Part H: Results, evidence and submission . . . 42
Step 38: The final run (results + screenshots in one go) . . . 42
Step 39: Fill in the test-case document . . . 42
Step 40: Read a failure and document it . . . 42
Step 41: Pack the submission . . . 43
Part I: Exam day . . . 44
Step 42: The evening before and the morning of the exam . . . 44
Step 43: Time plan for 1 hour 45 minutes . . . 44
Part J: Viva questions and answers . . . 45
Part K: Troubleshooting . . . 47
Appendix A: Command card . . . 48
Appendix B: Locator reference (most used data-testids) . . . 49
Appendix C: Selenium cheat sheet (Python) . . . 50

Before you start: how to use this guide

You will mostly do three things: **install programs**, **type short commands** into a window called **PowerShell**, and **read and run test code**. Every step tells you exactly what to type and what you should see afterwards. The coloured boxes mean:

Type this, then press Enter

```
this is a command you type
```

You should see

This is what the screen should show when the step worked.

If something goes wrong

- This tells you how to fix the most common problems for that step.

Tip

Extra advice that makes things easier.

Exam tip

Something examiners look for, or a point to say in the viva.

Golden rules for typing commands

1. Type (or copy and paste) each command **exactly**: same spelling, spaces, dots and backslashes (\).
2. Press **Enter** after each command. Nothing happens until you do.
3. **Wait** until the command finishes: a new line starting with `PS C:\...>` appears.
4. Do one command at a time, in the order shown. Do not skip steps.
5. If a command shows red text, stop and read the *If something goes wrong* box of that step.
6. Copy and paste: select the text in this PDF, press Ctrl+C, click in PowerShell, press Ctrl+V (or right-click), then Enter. **Python code** is different: open the files in the project instead of copying code from the PDF, because a PDF can lose the spaces at the start of lines, and Python needs them.

Words you will see

Word	Meaning
PowerShell	A window where you type commands to the computer (comes with Windows).
Virtual environment (.venv)	A private folder with the project's own copy of Python packages.
Server / app	The running Defect Prediction website. It must stay running while you use it in a browser.
Selenium WebDriver	A Python library that controls a real Chrome browser from code: open pages, type, click.
chromedriver	A small helper program between Selenium and Chrome. Selenium downloads it for you.
pytest	The tool that finds and runs test functions and reports PASSED / FAILED.
Test case	One check with an ID, steps, test data and an expected result.
Assertion	A line `assert actual == expected`. If it is false, the test FAILS.
Locator	How the test finds an element, e.g. `[data-testid="login-email-input"]`.
Page Object	A Python class that holds the locators and actions for one page (reusable steps).
Fixture	Setup code pytest runs for a test automatically, e.g. opening the browser.

Word	Meaning
Positive / negative test	Positive = valid input, expect success. Negative = invalid or empty input, expect a clear error and NO success.

PART A

Set up the tools and the application

Do this once, ideally the day before the exam. At the end the app runs on your laptop and you have logged in by hand.

1 Step 1: Open PowerShell

~1 min

1. Click **Start** (Windows logo) or press the Windows key.
2. Type **PowerShell**.
3. Click **Windows PowerShell** in the results. A blue or black window opens.

You should see
A line like `PS C:\Users\YourName>` and a blinking cursor. Keep this window open.

Tip
Do **not** choose "Run as administrator". A normal window is all you need. On some computers the app is called **Terminal**; that works too.

2 Step 2: Install Python 3.11

~5 min

The project needs **Python 3.11 exactly**. Its machine-learning models and pinned packages are built for 3.11. Other versions can stay installed next to it.

1. Open `https://www.python.org/downloads/release/python-3119/` in your browser.
2. Scroll to **Files** and click **Windows installer (64-bit)**. The file python-3.11.9-amd64.exe downloads.
3. Open the downloaded file.
4. **Important:** on the first screen tick **"Add python.exe to PATH"**.
5. Click **Install Now**. If Windows asks "Do you want to allow this app to make changes?", click **Yes**.
6. When you see **Setup was successful**, click **Close**.
7. Close PowerShell and open it again (Step 1), so it notices the new program.

Check it worked

```
py -3.11 --version
```

You should see something like

```
Python 3.11.9
```

If something goes wrong

- **'py' is not recognized**: run the installer again, tick "Add python.exe to PATH" and keep "py launcher" selected. Then reopen PowerShell.
- **No suitable Python runtime found**: Python 3.11 is not installed (you have another version). Install 3.11.9 as above.
- **"Windows protected your PC"** when opening the installer: click **More info** then **Run anyway** (only for files from python.org).

3 Step 3: Install Google Chrome and check its version

~3 min

Selenium drives a real browser. These tests use **Google Chrome**. If Chrome is already installed, just check that it is up to date.

1. If needed, download Chrome from `https://www.google.com/chrome/` and install it with the default options.
2. Open Chrome, type `chrome://settings/help` in the address bar and press Enter.
3. Wait until it says **Chrome is up to date** (click **Relaunch** if it asks).

> **Tip**
> You do **not** need to download chromedriver yourself. Since Selenium 4.6, **Selenium Manager** downloads the driver that matches your Chrome automatically the first time a test runs (internet needed once).

4 Step 4: (Optional) Install Git

~3 min

Git is only needed for Option B in Step 5. If you will download the ZIP file, skip this step.

1. Go to `https://git-scm.com/download/win` and download the 64-bit installer.
2. Open it, click **Next** on every screen without changing anything, then **Install** and **Finish**.
3. Close PowerShell and open it again.

> **Check it worked**
> `git --version`

> **You should see**
> `git version 2.xx.x.windows.x`

5 Step 5: Download the project

~5 min

Option A (recommended): download the ZIP file

1. Open `https://github.com/Debdeeptax/defect-predictor` (no GitHub account needed: the project is public).
2. Click the green **<> Code** button, then **Download ZIP**. The file **defect-predictor-master.zip** downloads (about 45 MB).
3. Open your **Downloads** folder. **Right-click** the ZIP and choose **Extract All...**
4. In the box "Files will be extracted to this folder", delete what is there and type `C:\SDP`. Click **Extract**.
5. A window shows the folder **defect-predictor-master**. Right-click it, choose **Rename**, type `defect-predictor` and press Enter.

> **You should see**
> The folder `C:\SDP\defect-predictor` exists and contains run.py, requirements.txt, README.md and the folders app, data, models and tests.

Option B: download with Git

Type these, pressing Enter after each line

```
mkdir C:\SDP
cd C:\SDP
git clone https://github.com/Debdeeptax/defect-predictor.git defect-predictor
```

You should see

The download finishes with a line like `Resolving deltas: 100%`.

If something goes wrong

- **A folder named defect-predictor already exists**: you downloaded it before. Use that folder, or delete it and try again.

6 Step 6: Go into the project folder and check the test files

~1 min

All commands from now on are typed **inside the project folder**. Repeat this step every time you open a new PowerShell window.

Type these, pressing Enter after each line

```
cd C:\SDP\defect-predictor
dir tests\selenium\tests
```

You should see

The line now starts with `PS C:\SDP\defect-predictor>`, and the list shows files including:

```
conftest.py
test_exam_01_login.py ... test_exam_07_my_scenario.py
test_login.py   test_role_access.py   test_upload_predict.py
```

If something goes wrong

- **Cannot find path**: the folder has a different name or place. In File Explorer find the folder that contains run.py and check it is `C:\SDP\defect-predictor`.
- **You see only one folder called defect-predictor-master inside**: the ZIP was extracted into an extra folder. Move its contents up one level.
- **The test_exam_ files are missing**: your copy is older than this guide. Download the ZIP again (Step 5).

7 Step 7: Create the project's private Python folder (virtual environment)

~1 min

Type this, then press Enter

```
py -3.11 -m venv .venv
```

You should see

No message, just a new command line after a few seconds. A folder named **.venv** now exists in the project folder. That is correct.

Tip

This guide always runs Python as `.\.venv\Scripts\python.exe`. That uses the project's own Python directly, so you never need to "activate" anything and PowerShell security settings cannot block you.

8 Step 8: Install the project's packages

5-15 min

Type these, one at a time, and wait for each to finish

```
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install pytest-html==4.2.0
```

You should see

Each command ends with `Successfully installed ...` . The second one takes the longest (it installs Flask, scikit-learn, SHAP, Selenium, pytest and more). `pytest-html` adds the HTML test report used for your submission.

Check it worked

```
.\.venv\Scripts\python.exe -c "import selenium, pytest; print('selenium', selenium.__version__, '| pytest', pytest.__version__)"
```

You should see

```
selenium 4.52.0 | pytest 8.4.2
```

If something goes wrong

- **It seems stuck**: large packages download silently. Wait up to 15 minutes before doing anything.
- **Red ERROR about a network or timeout**: check the internet (a college network may block it; try a mobile hotspot) and run the same command again.
- **ERROR: Could not find a version that satisfies...**: the .venv was made with the wrong Python. Delete the .venv folder and repeat Step 7 with `py -3.11` .

9 Step 9: Create the settings file (.env)

~1 min

The app reads its settings from a file named **.env**. Copy the example file and give it a random secret key (used to protect logins).

Type these, pressing Enter after each line

```
copy .env.example .env
$key = .\.venv\Scripts\python.exe -c "import secrets; print(secrets.token_hex(32))"
(Get-Content .env) -replace '^SECRET_KEY=.*', "SECRET_KEY=$key" | Set-Content .env
```

Check it worked

```
Select-String -Path .env -Pattern SECRET_KEY
```

You should see

A line `SECRET_KEY=` followed by 64 random letters and digits (not "change-me").

10 Step 10: Create the database with demo users and a sample report

~1 min

Type this, then press Enter

```
.\.venv\Scripts\python.exe scripts\reset_db.py
```

You should see

```
Database reset.
created developer dev@test.com / Test@1234
created qa        qa@test.com / Test@1234
created admin     admin@test.com / Test@1234
created sample report #1 for dev@test.com (8 files, 1 medium, 0 high)
```

Tip

Run this again whenever you want to start from a clean state. It **deletes** everything created in the app since (your own uploads, extra users). The automated tests in Part D do **not** need it: they make their own temporary database.

11 Step 11: Start the app and log in by hand

~2 min

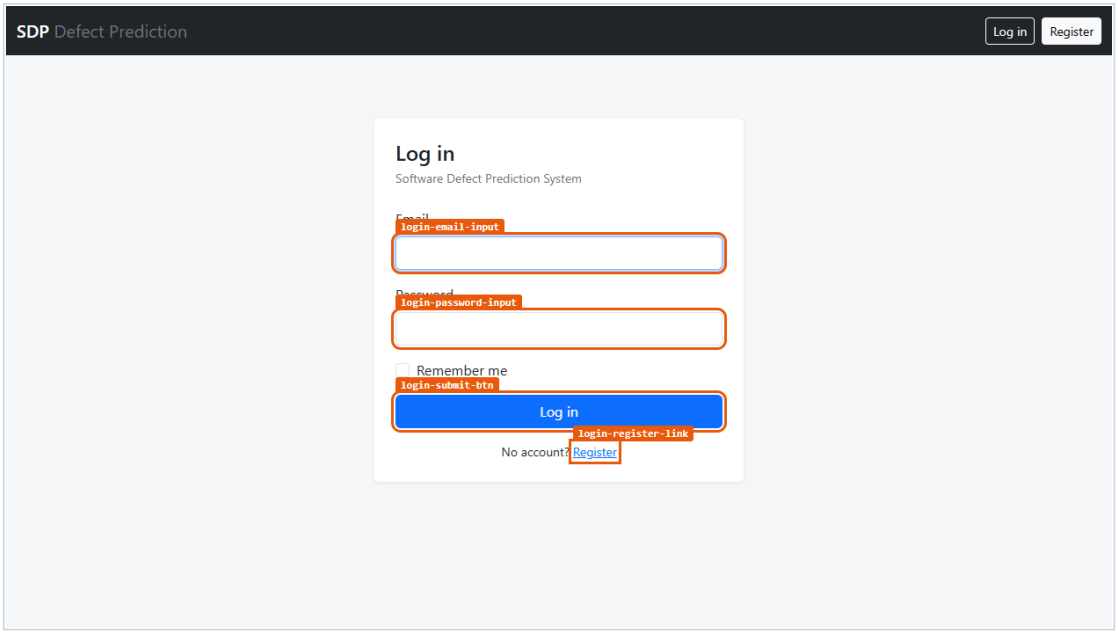
Type this, then press Enter

```
.\.venv\Scripts\python.exe run.py
```

You should see

```
Software Defect Prediction System is running.
Open http://127.0.0.1:5000 in your web browser.
Keep this window open while you use the app. Press CTRL+C to stop it.
```

1. Open Chrome and go to `http://127.0.0.1:5000`.
2. Log in with email `dev@test.com` and password `Test@1234`.
3. You land on the **Dashboard**. Click **My reports** and open **Sample project (seed)**.



The login page. The orange labels show the data-testid of each element; you will use them as locators in Part C.

If something goes wrong

- **This site can't be reached**: the PowerShell window running run.py was closed or shows an error. Start it again.
- **Address already in use / port 5000**: another copy is running. Close the other PowerShell window, or start with `$env:PORT="5001"; .\.venv\Scripts\python.exe run.py` and open port 5001.
- **Invalid email or password**: run Step 10 again, then log in.

12 Step 12: Stop the app (and start it again on another day)

~1 min

1. Click in the PowerShell window that runs the app and press **Ctrl+C**. The command line returns.
2. **Next time**: open PowerShell, do Step 6 (`cd C:\SDP\defect-predictor`), then Step 11 (`.\.venv\Scripts\python.exe run.py`). That is all.

Exam tip

Setup is finished. The exam sheet asks you to **verify the setup beforehand**: Step 18 (first script) and Step 19 (all tests pass) are exactly that check. Do them the evening before the exam.

PART B

Know the application you are testing

You cannot test what you do not understand. Spend 10 minutes clicking through the app with the three demo accounts. Testers call this *exploratory testing*, and it tells you what the expected results are.

13 Step 13: Learn the three roles

~3 min

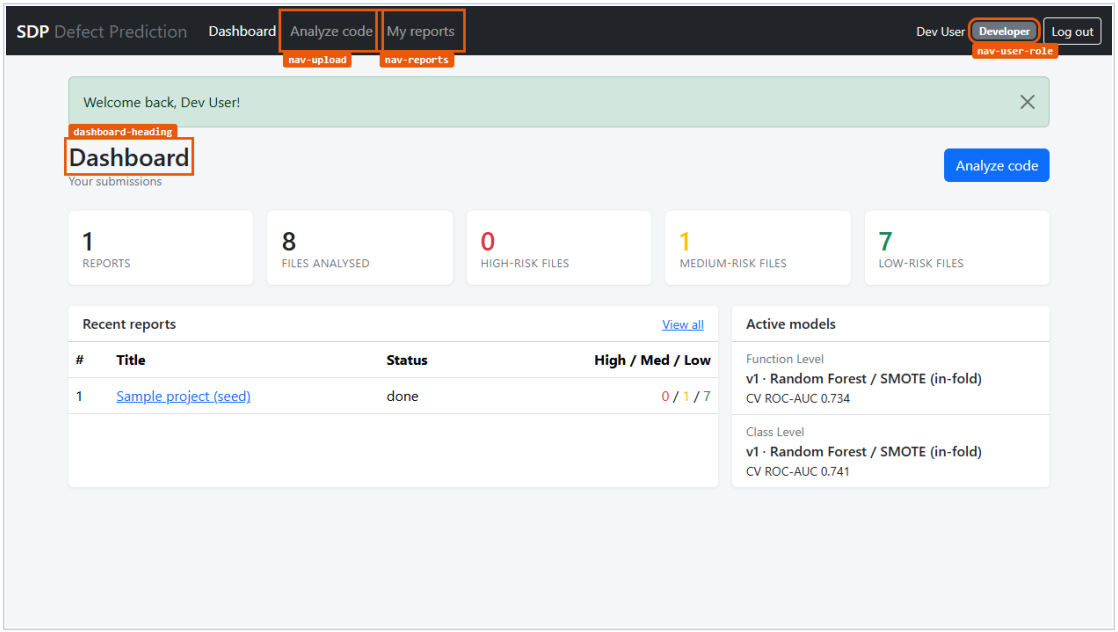
Role	Demo login (password Test@1234)	Can do	Menu items
Developer	dev@test.com	Analyze code (upload), see own reports	Dashboard, Analyze code, My reports
QA Engineer	qa@test.com	See all reports, compare two reports. Cannot upload	Dashboard, All reports, Compare runs
Admin	admin@test.com	Everything, plus users, models and logs	Dashboard, Analyze code, All reports, Compare runs, Models, Users, Logs

Anyone can **register** a new account; new accounts are Developers. Opening a page your role may not use shows an **Access denied** page with the code **403**.

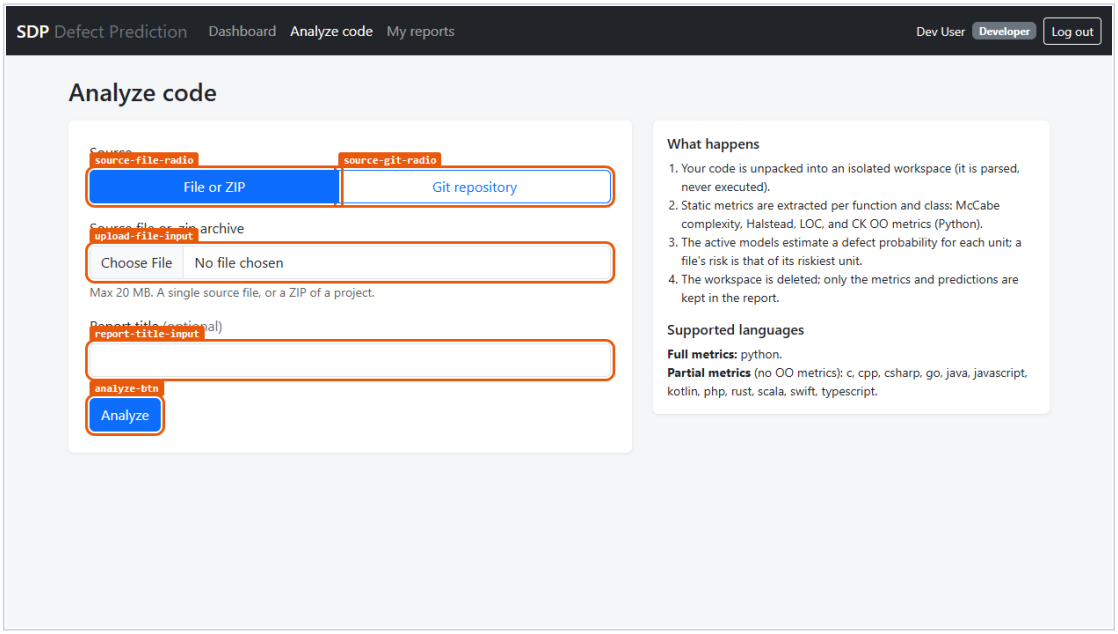
14 Step 14: Take the guided tour

~7 min

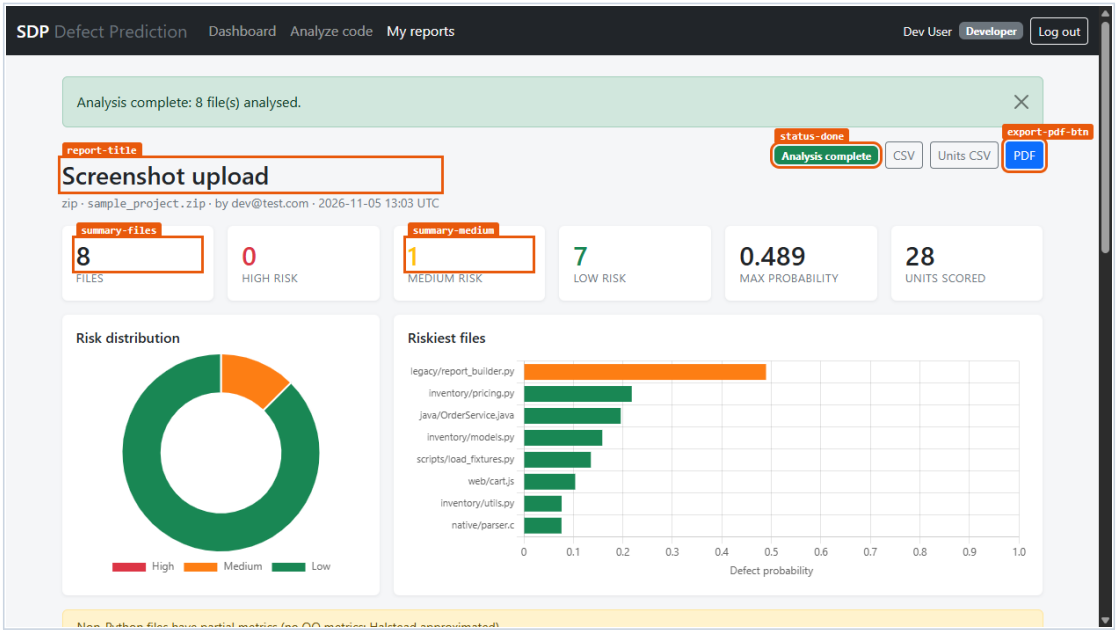
Log in as each user and open these pages. The screenshots show the elements the tests use.



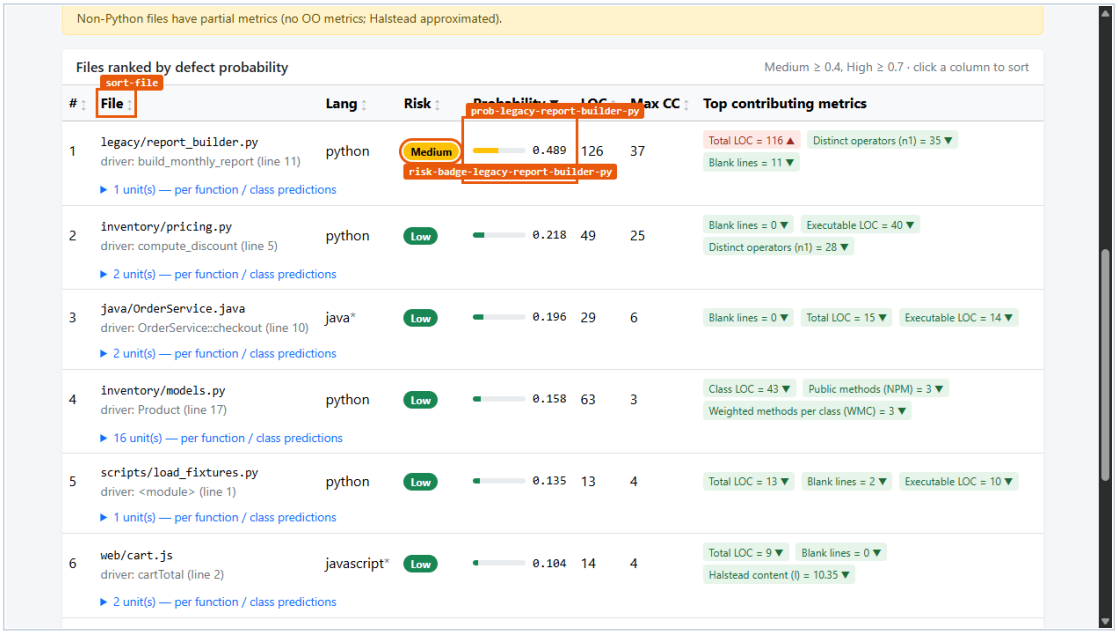
Dashboard (Developer). The role badge (nav-user-role) tells the tests who is logged in.



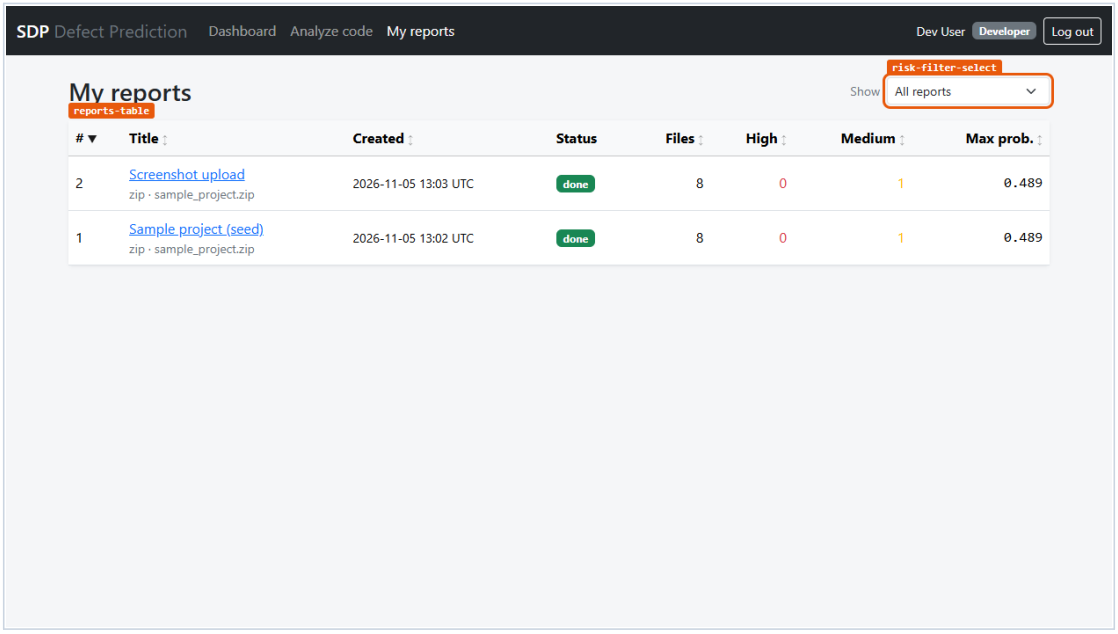
Analyze code: choose a file or a ZIP (or a Git URL), optionally a title, then Analyze.



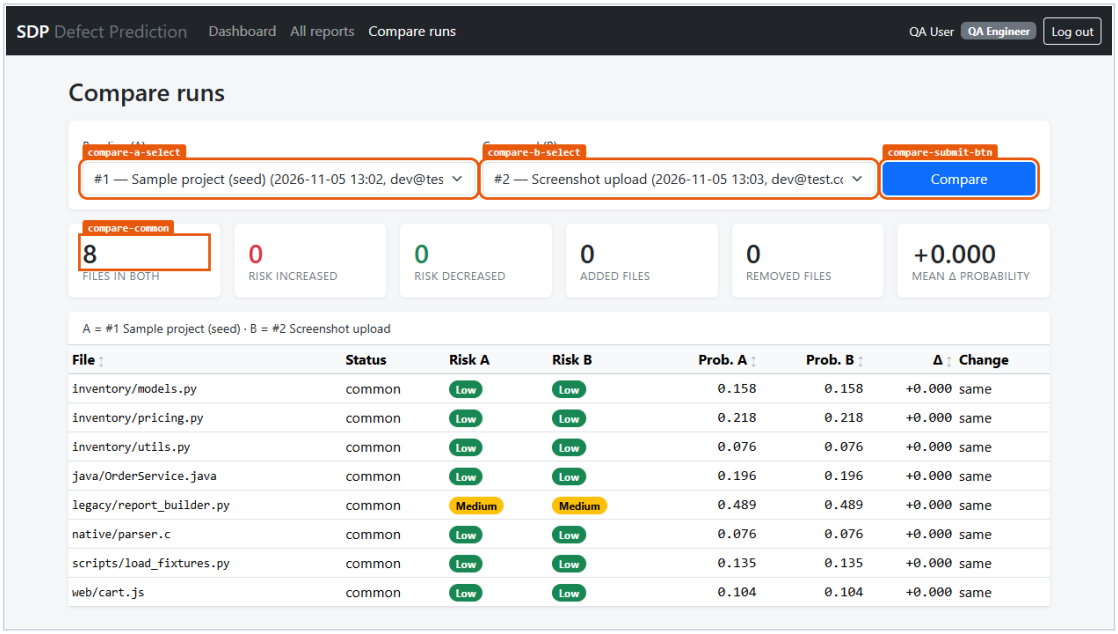
Report page after an analysis: title, status marker, summary cards and export links.



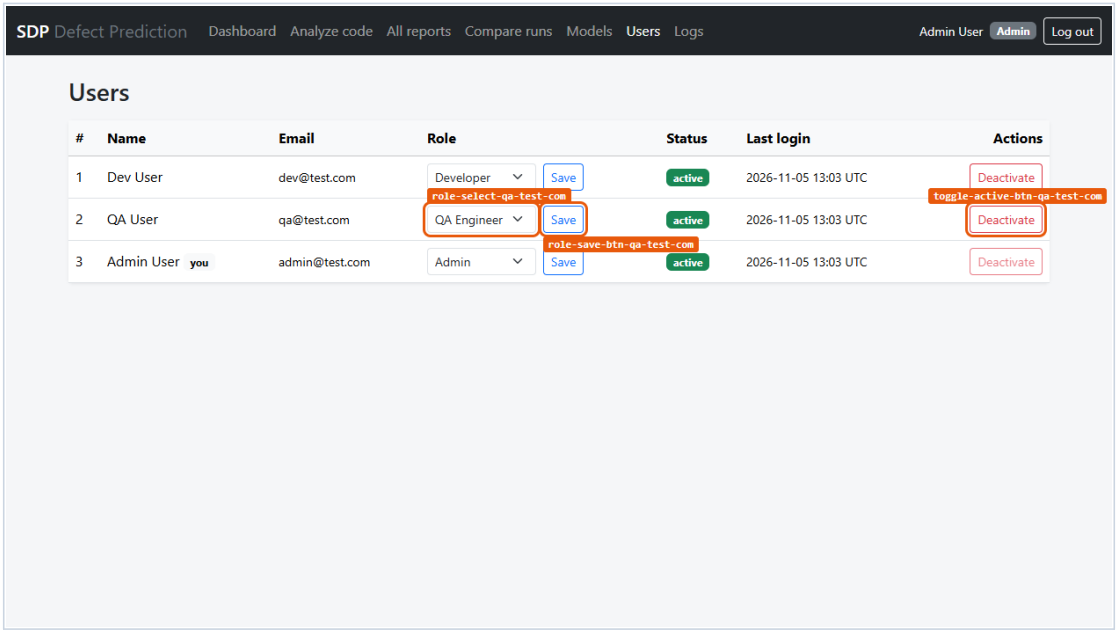
The ranked file table: risk badges, probabilities, sortable columns and a per-file drill-down ("unit(s)").



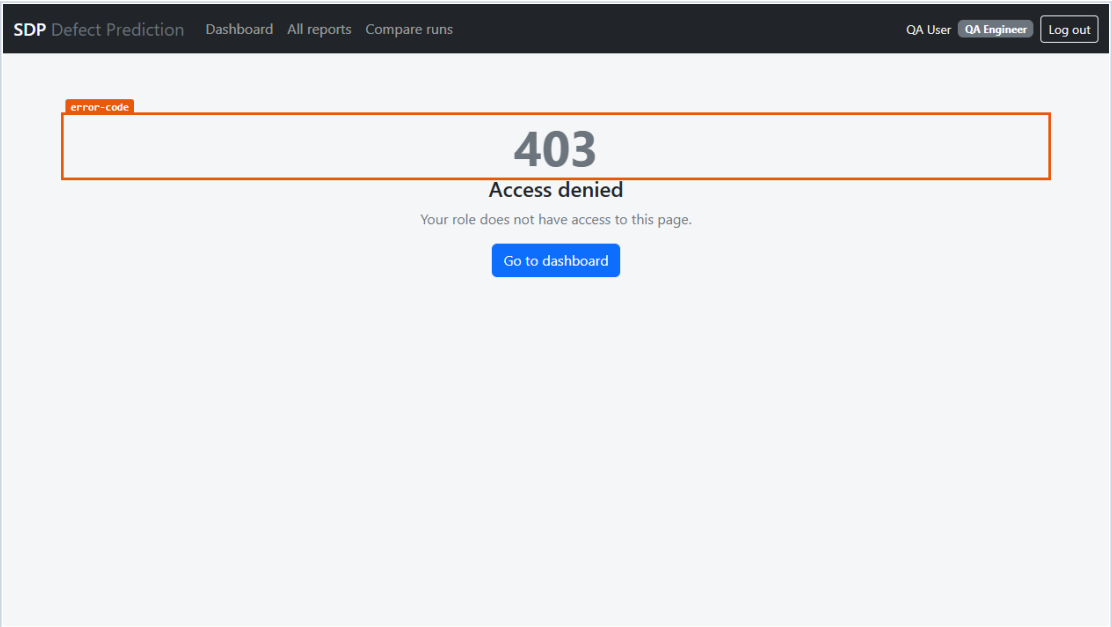
Reports list with the risk filter dropdown (a element).



Compare runs (QA, Admin): two dropdowns and the comparison result.



Admin, Users: change a role with a dropdown, activate or deactivate accounts.



Access denied: what a QA Engineer sees when opening /upload directly.

15 Step 15: Write down the exact messages (your expected results)

~5 min

Assertions compare text **exactly**, including full stops and capital letters. These are the real messages of this app (all checked by the tests in Part F):

Action	Exact message shown
Login with wrong password or unknown email	Invalid email or password.
Login with an empty email or password	Please enter a valid email and password.
Login to a deactivated account	This account has been deactivated. Contact an administrator.
Successful login	Welcome back, Dev User! (green)
Log out	You have been logged out.
Successful registration	Account created. You can log in now.
Register with an existing email	An account with this email already exists.
Register with different passwords	Passwords must match.
Password shorter than 8 characters	Field must be between 8 and 128 characters long.
Password with only letters or only digits	Password must be at least 8 characters and contain letters and digits.
Register with an invalid email	Please enter a valid email address.
Register with an empty field	This field is required.
Upload: no file chosen	Please choose a file to upload.
Upload: a .txt file	'notes.txt' is not a supported source file. Upload a source file or a .zip archive.
Upload: an empty file	The uploaded file is empty.
Git source with no URL	Please enter a Git repository URL.
Git URL on another website	Host 'example.com' is not allowed. Allowed: github.com, gitlab.com, bitbucket.org.
Successful upload of the sample ZIP	Analysis complete: 8 file(s) analysed.

Action	Exact message shown
Compare a report with itself	Please choose two different reports.
Opening a page your role may not use	403, Access denied

Exam tip

In the exam, do this first for the scenario you are given: use the app by hand with valid, invalid and empty input and **copy every message exactly**. Those become the expected results of your test cases.

PART C

Selenium basics in 15 minutes

Just enough theory to understand every line of the tests, then your first script.

16 Step 16: Understand how a Selenium test works

~4 min

1. Your test (Python)	2. Selenium library	3. chromedriver	4. Google Chrome	5. The app (Flask)
test_exam_01_login.py says: open /login, type, click	turns it into WebDriver commands	receives them over HTTP (W3C WebDriver protocol)	really opens the page, types and clicks	answers like for a human user

Every automated test, however long, does the same four things, often described as **Arrange, Act, Assert**:

1. **Open** a page (Arrange: get to the starting point, e.g. log in).
2. **Perform actions**: type, click, choose from a dropdown, upload a file (Act).
3. **Verify** the outcome with `assert` (Assert). No assert means it is not a test.
4. **Record** the result: PASSED / FAILED, plus a screenshot as evidence.

pytest finds every function whose name starts with `test_` in files named `test_*.py`, runs them one by one and prints PASSED or FAILED for each.

17 Step 17: Find elements: locators and the data-testid attribute

~6 min

To click or type, the test must first **find** the element. In this app every button, input, link, message and table row has a stable **data-testid** attribute (and the same value as its id), for example:

```
<input type="email" id="login-email-input" data-testid="login-email-input" ...>
```

See it yourself: open the login page in Chrome, right-click the email box, choose **Inspect** (or press F12). The HTML of that element is highlighted. Then test a locator in the **Console** tab:

> **Type this in Chrome DevTools > Console (not PowerShell)**
> `$$('[data-testid="login-email-input"]')`

> **You should see**
> A list with exactly **one** element. Zero means the locator is wrong; more than one means it is not unique.

Strategy	Example for this app	When to use
data-testid (CSS)	`By.CSS_SELECTOR, '[data-testid="login-submit-btn"]'`	First choice here: made for tests, never changes with styling
ID	`By.ID, "login-submit-btn"`	Unique and fast. Good when there is no data-testid
NAME	`By.NAME, "email"`	Form fields
CSS selector	`By.CSS_SELECTOR, "#login-form button[type='submit']"`	Combinations, attributes
XPath	`By.XPATH, "//button[text()='Log in']"`	Last choice: by visible text or relations; long XPaths break easily
LINK_TEXT	`By.LINK_TEXT, "Register"`	Links by their exact text

In the project the helper `tid("login-email-input")` builds the data-testid locator for you, and the full list of every data-testid is in `docs\selenium-locators.md` (Appendix B lists the most important ones).

> **Exam tip**
> A favourite viva question: *"Which locator did you use and why?"* Answer: data-testid, because it is a dedicated, unique, stable test hook. IDs and CSS classes can change when the page is redesigned; XPath based on the page structure breaks with any layout change.

18 Step 18: Write and run your first Selenium script

~5 min

This short script (no pytest yet) logs in as the Developer and checks the result. It is already in the project as `tests\selenium\selenium_first_script.py`. Read it line by line:

tests\selenium\selenium_first_script.py

```
r"""
YOUR FIRST SELENIUM SCRIPT (no pytest, no page objects) for the Defect Prediction System.

Before running: start the app in another PowerShell window with  .\.venv\Scripts\python.exe run.py
Run:            .\.venv\Scripts\python.exe tests\selenium\selenium_first_script.py

It does the 4 things every automated test does:
  1. OPEN a page   2. PERFORM actions   3. VERIFY the result   4. RECORD the result
"""
import os
from pathlib import Path

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

APP_URL = os.getenv("SDP_BASE_URL", "http://127.0.0.1:5000")

options = webdriver.ChromeOptions()
# Stop Chrome's "save password?" pop-up from stealing keyboard focus after logging in
options.add_experimental_option("prefs", {
    "credentials_enable_service": False,
    "profile.password_manager_enabled": False,
    "profile.password_manager_leak_detection": False,
})
if os.getenv("SDP_HEADLESS") == "1":
    options.add_argument("--headless=new")

driver = webdriver.Chrome(options=options)     # opens Chrome (chromedriver is downloaded automatically)
wait = WebDriverWait(driver, 15)               # explicit wait: up to 15 seconds

try:
    # 1. OPEN the login page
    driver.get(APP_URL + "/login")
    print("Opened:", driver.title)

    # 2. PERFORM ACTIONS - find each element by its data-testid and use it
    driver.find_element(By.CSS_SELECTOR, '[data-testid="login-email-input"]').send_keys("dev@test.com")
    driver.find_element(By.CSS_SELECTOR, '[data-testid="login-password-input"]').send_keys("Test@1234")
    driver.find_element(By.CSS_SELECTOR, '[data-testid="login-submit-btn"]').click()

    # 3. VERIFY - wait for the dashboard heading, then check the URL and the role badge
    heading = wait.until(EC.visibility_of_element_located(
        (By.CSS_SELECTOR, '[data-testid="dashboard-heading"]')))
    role = driver.find_element(By.CSS_SELECTOR, '[data-testid="nav-user-role"]').get_attribute("data-role")
    assert driver.current_url.endswith("/dashboard"), driver.current_url
    assert role == "developer", role

    # 4. RECORD
    Path("reports").mkdir(exist_ok=True)
    driver.save_screenshot("reports/first_script_PASS.png")
    print("Heading:", heading.text, "| role:", role)
    print("RESULT: PASS - screenshot saved to reports\\first_script_PASS.png")
```

tests\selenium\selenium_first_script.py

```
55  except Exception as error:
56      Path("reports").mkdir(exist_ok=True)
57      driver.save_screenshot("reports/first_script_FAIL.png")
58      print("RESULT: FAIL -", error)
59      raise
60  finally:
61      driver.quit()                           # always close the browser
```

It needs the app running, so use **two** PowerShell windows:

1. **Window 1**: Step 6 (`cd C:\SDP\defect-predictor`), then start the app: `.\.venv\Scripts\python.exe run.py` . Leave it running.
2. **Window 2**: open a second PowerShell, go to the folder again, and run the script:

Type this, then press Enter

```
cd C:\SDP\defect-predictor
.\.venv\Scripts\python.exe tests\selenium\selenium_first_script.py
```

You should see

Chrome opens by itself, logs in and closes again. Window 2 shows:

```
Opened: Log in – SDP
Heading: Dashboard | role: developer
RESULT: PASS - screenshot saved to reports\first_script_PASS.png
```

If something goes wrong

- **ERR_CONNECTION_REFUSED**: the app is not running. Start it in Window 1.
- **SessionNotCreatedException** or Chrome opens and closes at once: update Chrome (Step 3) and make sure you are online (the driver is downloaded on first use).
- **TimeoutException**: login did not reach the dashboard. Run Step 10 again to restore the demo users.

Line(s)	What it does
`webdriver.Chrome(options=options)`	Starts Chrome. The options switch off Chrome's password pop-up, which can steal keyboard focus after a login.
`WebDriverWait(driver, 15)`	An **explicit wait**: wait up to 15 s for a condition, continue as soon as it is true.
`driver.get(...)`	Opens a URL.
`find_element(...).send_keys(...)`	Finds an element and types into it.
`.click()`	Clicks the element.
`wait.until(EC.visibility_of_element_located(...))`	Waits until the dashboard heading is visible (the next page has loaded).
`get_attribute("data-role")`	Reads an HTML attribute (here: which role is logged in).
`assert ...`	The checks. A false assert raises AssertionError: the result is FAIL.
`save_screenshot(...)`	Evidence.
`finally: driver.quit()`	Always closes Chrome, even after a failure.

Stop the app in Window 1 with **Ctrl+C**. The real tests in Part D start their own copy of the app.

PART D

Run the project's automated tests

The project contains 70 Selenium tests: 16 original sample tests and 54 exam-oriented tests (Part F). You do not need run.py for these: each test run starts its own private copy of the app.

19 Step 19: Run the whole Selenium test suite

~4 min

Type this, then press Enter

```
.\.venv\Scripts\python.exe -m pytest tests\selenium -v
```

You should see

One line per test ending in PASSED, then the summary (the time varies):

```
tests/selenium/tests/test_exam_01_login.py::test_login_data_driven[TC_LOGIN_01_valid_developer] PASSED
...
======================== 70 passed in 189.76s (0:03:09) ========================
```

What happened: pytest started the app in the background on a free port with a **fresh temporary database** (the three demo users and report #1), opened one Chrome window **without showing it** (headless), ran every test, and closed everything. Your own database from Step 10 is not touched.

If something goes wrong

- **SKIPPED ... Chrome / chromedriver not available**: Chrome is missing or cannot start. Do Step 3 and make sure you are online for the first run.
- **ModuleNotFoundError: No module named 'selenium'**: you typed `python` or `pytest` instead of `.\.venv\Scripts\python.exe -m pytest`.
- **Some tests FAILED**: read Step 40 (how to read a failure). Run the failed file again on its own first; a slow laptop can hit a timeout once.

Exam tip

The exam sheet asks you to verify your setup beforehand. This command passing is that proof. Take a screenshot of the summary line the evening before.

20 Step 20: Watch the browser while the tests run

~3 min

By default the browser is invisible (headless), which is faster. To **see** the clicks and typing, for learning or for the examiner:

Type these, pressing Enter after each line

```
$env:SDP_HEADLESS="0"
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_01_login.py -v
```

You should see

A Chrome window appears with the bar *"Chrome is being controlled by automated test software"* and logs in again and again, much faster than a human.

Switch back to invisible mode

```
Remove-Item Env:SDP_HEADLESS
```

Tip
The setting only lasts for this PowerShell window. Other settings work the same way: `$env:SDP_TIMEOUT="60"` gives slow laptops 60 s instead of 30 s for every wait.

21 Step 21: Run only part of the tests

~2 min

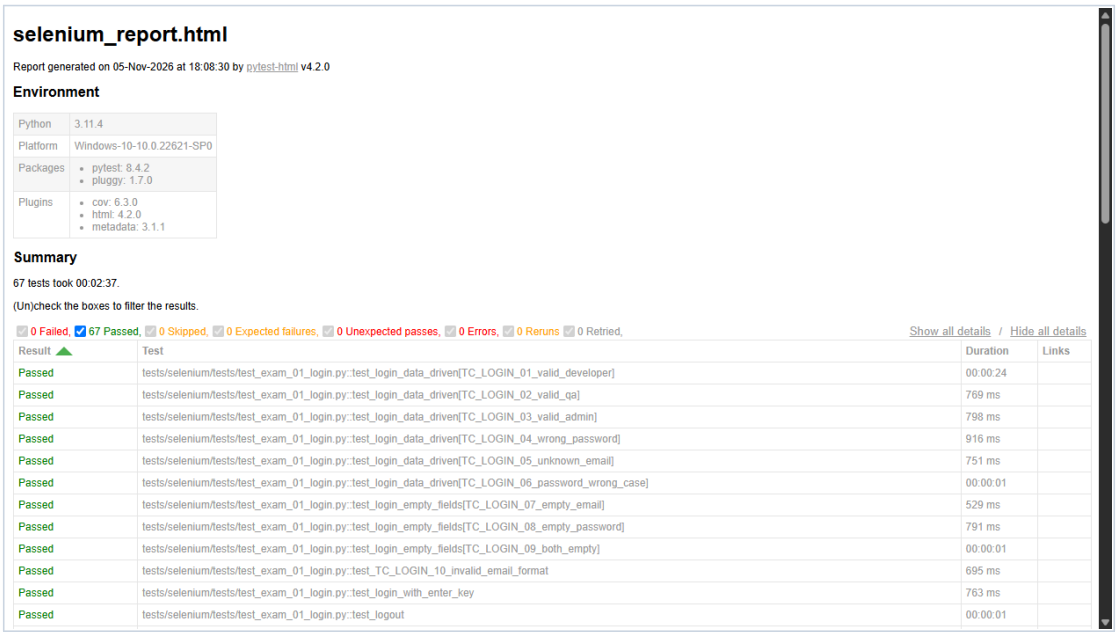
You want to run	Command (after `.\.venv\Scripts\python.exe -m pytest`)
One file	`tests\selenium\tests\test_exam_03_upload.py -v`
One test function	`tests\selenium\tests\test_exam_01_login.py::test_logout -v`
Tests whose name contains a word	`tests\selenium -k upload -v`
Everything except some	`tests\selenium -k "not compare" -v`
Stop at the first failure	`tests\selenium -x`
Show print() output	`tests\selenium\tests\test_exam_01_login.py -s`
List the tests without running	`tests\selenium --collect-only -q`

22 Step 22: Create the HTML test report

~4 min

Type this, then press Enter
```
.\.venv\Scripts\python.exe -m pytest tests\selenium -v --html=reports\selenium_report.html --self-contained-html
```

Open the report in your browser
```
start reports\selenium_report.html
```



The pytest-html report: environment, summary (70 passed), and one row per test. Click a row to see its screenshot and details.

The file is **self-contained**: the screenshots are inside it, so it can be e-mailed or uploaded as one file. Failed tests are shown in red with the error message.

23 Step 23: Find the screenshot evidence

~2 min

A small file `tests\selenium\tests\conftest.py` (explained in Step 26) saves a screenshot after **every** test, named after the result and the test:

Type this, then press Enter

```
dir tests\selenium\artifacts\evidence
```

You should see

```
PASS_test_login_data_driven_TC_LOGIN_01_valid_developer__101530.png
PASS_test_TC_UPL_01_upload_zip_shows_ranked_report_101612.png  ...
```

When a test **fails**, the project's main `tests\selenium\conftest.py` also saves the screenshot **and the page's HTML** in `tests\selenium\artifacts` to help you find the cause.

Before your final run, delete old evidence (an error "cannot find path" just means it is already empty)

```
Remove-Item -Recurse -Force tests\selenium\artifacts\evidence
```

24 Step 24: (Optional) Two extra ways to run tests

~5 min

a) Test the app you started yourself

Normally the tests start their own app. To run them against the app from Step 11 instead (useful to watch the data appear in your own browser), reset the demo data, start the app in Window 1, then in Window 2:

Type these, pressing Enter after each line

```
$env:SDP_BASE_URL="http://127.0.0.1:5000"
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_01_login.py -v
Remove-Item Env:SDP_BASE_URL
```

b) Run the whole project test suite (unit + integration + Selenium)

Type this, then press Enter

```
.\.venv\Scripts\python.exe -m pytest -q
```

You should see

```
230 passed in 269.37s (0:04:29)
```

PART E

How the test framework is built

Read this part with the files open in an editor (Notepad works; VS Code is nicer). Examiners ask *"explain your framework"*; this part is the answer.

25 Step 25: The folder map

~3 min

```
Structure of the Selenium tests
tests\selenium\
|-- conftest.py                fixtures: base_url (own app), _browser (Chrome), driver, timeout
|                              + saves screenshot and HTML of FAILED tests in artifacts\
|-- selenium_first_script.py   Step 18: the plain script without pytest
|-- data\
|   `-- login_data.csv         test data for the data-driven login test
|-- pages\                     PAGE OBJECTS (reusable steps)
|   |-- base_page.py           BasePage: open, find, click, type, wait, flash messages, logout ...
|   |-- login_page.py          LoginPage: login(), login_as(), error_message
|   |-- upload_page.py         UploadPage: select_file(), upload_file(), submit_expecting_error() ...
|   `-- report_page.py         ReportPage: summary(), file_paths(), sort_by(), expand_units() ...
|-- tests\                     THE TESTS
|   |-- conftest.py            evidence screenshot after every test + new_user fixture
|   |-- test_login.py, test_role_access.py, test_upload_predict.py   (original samples)
|   `-- test_exam_01_login.py ... test_exam_07_my_scenario.py         (Part F and G)
`-- artifacts\                 created when tests run: evidence\ and failure files
```

> **Tip**
> `pytest.ini` in the project folder tells pytest to look in `tests` and to treat the project folder as the import root, so tests can write `from tests.selenium.pages import LoginPage`.

26 Step 26: Fixtures: what happens before and after each test

~6 min

A **fixture** is setup code pytest runs for a test when the test names it as a parameter. The tests use four fixtures from `tests\selenium\conftest.py`:

Fixture	Scope	What it does
`base_url`	once per run	Builds a **separate copy of the app** in a temporary folder (own database, uploads, models), adds the demo users and report #1, starts it on a free port in a background thread, and returns its URL. Every run starts from the same known state, so results are repeatable. With `SDP_BASE_URL` set, it uses that server instead.
`_browser`	once per run	Starts Chrome once (headless unless SDP_HEADLESS=0, window 1400x1000, password manager off). If Chrome cannot start, the tests are SKIPPED with a clear message. Quits Chrome at the end.
`driver`	every test	Gives the test that browser after **deleting all cookies** and opening /login, so each test starts logged out and does not depend on the one before it.
`timeout`	every test	How long waits may take (30 s, or SDP_TIMEOUT).

The second conftest, `tests\selenium\tests\conftest.py`, adds the exam evidence and one reusable step:

```
tests\selenium\tests\conftest.py
1  """Evidence for the exam: save a screenshot after EVERY Selenium test (pass or fail)
2  into tests/selenium/artifacts/evidence/ and attach it to the pytest-html report."""
3  import uuid
4  from datetime import datetime
5  from pathlib import Path
6
```

tests\selenium\tests\conftest.py

```python
import pytest

EVIDENCE = Path(__file__).resolve().parents[1] / "artifacts" / "evidence"


@pytest.fixture
def new_user(driver, base_url, timeout):
    """REUSABLE STEP: register a brand-new Developer account through the UI and return
    its email (password Passw0rd1). Ends on the login page, logged out."""
    from tests.selenium.pages import BasePage
    email = f"user_{uuid.uuid4().hex[:8]}@test.com"
    page = BasePage(driver, base_url, timeout)
    page.open("/register")
    page.wait_for("register-form")
    page.type("register-name-input", "Exam User")
    page.type("register-email-input", email)
    page.type("register-password-input", "Passw0rd1")
    page.type("register-confirm-input", "Passw0rd1")
    page.click("register-submit-btn")
    page.wait_for("login-form")
    return email


@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when != "call" or "driver" not in item.fixturenames:
        return
    driver = item.funcargs.get("driver")
    if driver is None:
        return
    status = "PASS" if report.passed else "FAIL"
    name = "".join(c if c.isalnum() or c in "-_" else "_" for c in item.name)[:100]
    EVIDENCE.mkdir(parents=True, exist_ok=True)
    try:
        driver.save_screenshot(str(EVIDENCE / f"{status}_{name}_{datetime.now():%H%M%S}.png"))
        image = driver.get_screenshot_as_base64()
    except Exception:          # e.g. an alert is still open
        return
    html = item.config.pluginmanager.getplugin("html")
    if html is not None:       # pytest-html installed -> embed the screenshot in the report
        report.extras = getattr(report, "extras", []) + [html.extras.image(image, "Screenshot")]
```

`pytest_runtest_makereport` is a pytest **hook**: pytest calls it after each test. Here it saves a PASS_/FAIL_ screenshot and attaches it to the HTML report. `new_user` registers a fresh account through the UI, so tests that change a user (role, deactivation) never change the shared demo accounts.

27 Step 27: Page Object Model: the reusable steps

~8 min

A **Page Object** is a class for one page with its locators and actions. Tests call `page.login(email, password)` instead of repeating find/type/click everywhere. If the page changes, only the page object is updated. These are the methods you will use most:

Class	Method / property	What it does
BasePage	`open(path)`	Go to base_url + path and wait until the page is loaded
	`find(id) / exists(id)`	Find an element by data-testid / True if it is on the page
	`wait_for(id) / wait_visible(id)`	Wait until the element is present / visible
	`click(id) / type(id, text)`	Wait until clickable, click / clear the field, type, and check it stuck

Class	Method / property	What it does
	`text(id)`	Visible text of an element
	`flash_messages(category)`	Texts of the success / error / info messages at the top
	`user_role / is_logged_in`	'developer', 'qa' or 'admin' / True if Log out is shown
	`nav_items() / logout() / error_code`	Visible menu links / log out / '403' on the error page
	`current_path`	The URL path, e.g. '/dashboard'
LoginPage	`login(email, password)`	Fill in the form, click Log in, wait for the next page
	`login_as(email)`	Open /login, log in, check it worked, return a BasePage
	`error_message`	The first error message
UploadPage	`upload_file(path, title)`	Choose the file, set the title, Analyze, wait for the report
	`select_file(path) / submit_expecting_error()`	Choose a file / click Analyze and return the error
	`choose_git_source()`	Switch the source to Git repository
ReportPage	`open_report(id) / title / is_done`	Open /reports/id / its title / finished?
	`summary()`	Dict of the summary cards: files, high, medium, low, max-prob, units
	`file_paths() / risk_of(path) / probability_of(path)`	Table order / risk badge / probability
	`sort_by(column) / expand_units(path) / export_url(kind)`	Click a header / open the drill-down / link

Here is the core of `BasePage` and `LoginPage`. Notice that every action waits first:

```
From pages\base_page.py and pages\login_page.py (shortened)
def click(self, testid):
    self.wait.until(EC.element_to_be_clickable(tid(testid))).click()

def type(self, testid, text, clear=True):
    el = self.wait_visible(testid)
    if clear:
        el.clear()
    el.send_keys(text)

# LoginPage
def login(self, email, password):
    self.type("login-email-input", email)
    self.type("login-password-input", password)
    old = self.find("login-form")
    self.click("login-submit-btn")
    self.wait.until(lambda d: _is_stale(old))      # the old page is gone ...
    # ... and the new page is here: the dashboard (success) or an error message (failure)
    self.wait.until(lambda d: self.exists("dashboard-heading") or self.exists("flash-error"))
```

Tip

Why wait for the old form to become "stale"? After clicking, the old page is replaced. An element from the old page is then *stale*. Waiting for that proves the new page has arrived, so the test never reads the old page by mistake.

28 Step 28: Waits and sync markers: never use sleep

~4 min

Way of waiting	How	Verdict
`time.sleep(5)`	Always waits 5 s	Bad: slow, and still fails if the page needs 6 s
Implicit wait	`driver.implicitly_wait(10)`	Global retry for find_element only. Not used here
Explicit wait	`WebDriverWait(driver, 30).until(condition)`	**Used everywhere**: waits only as long as needed, for a specific condition

Common conditions: `presence_of_element_located`, `visibility_of_element_located`, `element_to_be_clickable`, `text_to_be_present_in_element`, `url_contains`, `alert_is_present`, `number_of_windows_to_be`. A wait that never comes true raises `TimeoutException`.

The app helps by showing **sync markers**: elements that appear only when work is finished. The upload test does not guess how long an analysis takes; it waits for `status-done` (or `status-failed` / `flash-error`) for up to 180 s. Charts get `data-rendered="true"` when drawn.

PART F

Exam scenarios: six complete test files

Each scenario has the test cases (what and why), the full code (already in the project), the command to run it, and what to say in the viva. Open each file next to this guide.

29 Step 29: Scenario 1: Login (data-driven, positive, negative, empty)

~20 min

Log in with valid accounts of every role, wrong credentials, empty fields and a badly formed email. The six main cases come from a CSV file, so **one test function runs six times** with different data.

TC ID	Type	Input	Expected result
TC_LOGIN_01-03	Positive	dev / qa / admin + Test@1234	Dashboard; role badge developer / qa / admin
TC_LOGIN_04	Negative	dev@test.com + Wrong@1234	Invalid email or password.; still logged out
TC_LOGIN_05	Negative	nobody@test.com + Test@1234	Invalid email or password.
TC_LOGIN_06	Negative	dev@test.com + test@1234 (wrong case)	Invalid email or password.
TC_LOGIN_07-09	Empty	email, password or both empty	Please enter a valid email and password.
TC_LOGIN_10	Negative	dev-at-test.com	Invalid email or password.
Enter key / logout	Positive	press Enter instead of clicking / click Log out	Dashboard / You have been logged out.

tests\selenium\tests\test_exam_01_login.py

```
"""EXAM SCENARIO 1 - LOGIN (positive, negative, empty, invalid format).
Data-driven: the rows come from tests/selenium/data/login_data.csv."""
import csv
from pathlib import Path

import pytest
from selenium.webdriver.common.keys import Keys

from tests.selenium.pages import LoginPage

DATA = Path(__file__).resolve().parents[1] / "data" / "login_data.csv"


def load_rows():
    with open(DATA, newline="", encoding="utf-8") as f:
        return [pytest.param(r["email"], r["password"], r["expected_result"], r["expected_message"],
                             id=r["case_id"]) for r in csv.DictReader(f)]


@pytest.mark.parametrize("email, password, expected_result, expected_message", load_rows())
def test_login_data_driven(driver, base_url, timeout, email, password, expected_result, expected_message):
    page = LoginPage(driver, base_url, timeout).open()          # 1. open the login page
    page.login(email, password)                                 # 2. type + click Login
    if expected_result == "success":                            # 3. verify
        assert page.current_path == "/dashboard"
        assert page.exists("dashboard-heading")
        assert page.user_role == expected_message               # role badge: developer / qa / admin
    else:
        assert page.error_message == expected_message
        assert not page.is_logged_in
        assert page.exists("login-form")


@pytest.mark.parametrize("email, password", [
    ("", "Test@1234"),
    ("dev@test.com", ""),
```

tests\selenium\tests\test_exam_01_login.py

```
    ("", ""),
], ids=["TC_LOGIN_07_empty_email", "TC_LOGIN_08_empty_password", "TC_LOGIN_09_both_empty"])
def test_login_empty_fields(driver, base_url, timeout, email, password):
    # The login form has `novalidate`, so the browser does NOT block empty fields:
    # the form is sent and the SERVER answers with an error message.
    page = LoginPage(driver, base_url, timeout).open()
    page.login(email, password)          # login() waits for the next page (dashboard or flash-error)
    assert page.error_message == "Please enter a valid email and password."
    assert page.current_path.startswith("/login")
    assert not page.is_logged_in


def test_TC_LOGIN_10_invalid_email_format(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).open()
    page.login("dev-at-test.com", "Test@1234")
    assert page.error_message == "Invalid email or password."
    assert not page.is_logged_in


def test_login_with_enter_key(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).open()
    page.type("login-email-input", "dev@test.com")
    page.find("login-password-input").send_keys("Test@1234", Keys.ENTER)   # press Enter instead of clicking
    page.wait_for("dashboard-heading")
    assert page.current_path == "/dashboard"


def test_logout(driver, base_url, timeout):
    dashboard = LoginPage(driver, base_url, timeout).login_as("dev@test.com")
    dashboard.logout()
    assert dashboard.exists("login-form")
    assert "You have been logged out." in dashboard.flash_messages("info")
```

Run it

```
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_01_login.py -v
```

You should see

```
test_login_data_driven[TC_LOGIN_01_valid_developer] PASSED ...
12 passed in 40.23s
```

Explain in the viva

The test data lives in `tests\selenium\data\login_data.csv` (shown below). `load_rows()` reads it and `@pytest.mark.parametrize` turns each row into its own test, named by the case_id, so the report shows every case separately. Adding a case means adding a CSV row, with no code change. Empty fields: the form has the HTML attribute *novalidate*, so the browser does not block it; the **server** answers with "Please enter a valid email and password.", and that is what we assert.

tests\selenium\data\login_data.csv

```
case_id,email,password,expected_result,expected_message
TC_LOGIN_01_valid_developer,dev@test.com,Test@1234,success,developer
TC_LOGIN_02_valid_qa,qa@test.com,Test@1234,success,qa
TC_LOGIN_03_valid_admin,admin@test.com,Test@1234,success,admin
TC_LOGIN_04_wrong_password,dev@test.com,Wrong@1234,error,Invalid email or password.
TC_LOGIN_05_unknown_email,nobody@test.com,Test@1234,error,Invalid email or password.
TC_LOGIN_06_password_wrong_case,dev@test.com,test@1234,error,Invalid email or password.
```

30 Step 30: Scenario 2: Registration and form validation

~15 min

Register a new account and log in with it (positive), then try every invalid input (negative) and leave each field empty in turn. The valid case uses a **unique email per run** (`uuid`), so the test can be run again and again.

TC ID	Type	Input	Expected result
TC_REG_01	Positive	valid name, new email, Passw0rd1 twice	Account created...; new user logs in as developer
TC_REG_02	Negative	email dev@test.com (exists)	An account with this email already exists.
TC_REG_03	Negative	Passw0rd1 / Passw0rd2	Passwords must match.
TC_REG_04	Negative	abc12 (5 characters)	Field must be between 8 and 128 characters long.
TC_REG_05-06	Negative	password / 12345678	Password must be at least 8 characters and contain letters and digits.
TC_REG_07	Negative	asha-at-test.com	Please enter a valid email address.
TC_REG_08-11	Empty	name, email, password or confirm empty	This field is required.

tests\selenium\tests\test_exam_02_register.py

```
"""EXAM SCENARIO 2 - REGISTRATION + FORM VALIDATION."""
import uuid

import pytest

from tests.selenium.pages import BasePage, LoginPage

WEAK = "Password must be at least 8 characters and contain letters and digits."


def unique_email():
    """A new address every run, so 'valid registration' never collides with an old account."""
    return f"student_{uuid.uuid4().hex[:8]}@test.com"


def fill_register_form(page, name, email, password, confirm):
    page.type("register-name-input", name)
    page.type("register-email-input", email)
    page.type("register-password-input", password)
    page.type("register-confirm-input", confirm)
    page.click("register-submit-btn")


@pytest.fixture
def register_page(driver, base_url, timeout):
    page = BasePage(driver, base_url, timeout)
    page.open("/register")
    page.wait_for("register-form")
    return page


def test_TC_REG_01_valid_registration_then_login(register_page, driver, base_url, timeout):
    email = unique_email()
    fill_register_form(register_page, "Asha Kumar", email, "Passw0rd1", "Passw0rd1")

    register_page.wait_for("login-form")                    # redirected to /login
    assert "Account created. You can log in now." in register_page.flash_messages("success")

    login = LoginPage(driver, base_url, timeout)
    login.login(email, "Passw0rd1")                         # the new account works
    assert login.current_path == "/dashboard"
    assert login.user_role == "developer"                   # new users are Developers


```

tests\selenium\tests\test_exam_02_register.py

```
@pytest.mark.parametrize("name, email, password, confirm, expected", [
    ("Asha", "dev@test.com", "Passw0rd1", "Passw0rd1", "An account with this email already exists."),
    ("Asha", "NEW", "Passw0rd1", "Passw0rd2", "Passwords must match."),
    ("Asha", "NEW", "abc12", "abc12", "Field must be between 8 and 128 characters long."),
    ("Asha", "NEW", "password", "password", WEAK),
    ("Asha", "NEW", "12345678", "12345678", WEAK),
    ("Asha", "asha-at-test.com", "Passw0rd1", "Passw0rd1", "Please enter a valid email address."),
], ids=["TC_REG_02_duplicate_email", "TC_REG_03_password_mismatch", "TC_REG_04_password_too_short",
        "TC_REG_05_password_letters_only", "TC_REG_06_password_digits_only", "TC_REG_07_invalid_email"])
def test_registration_server_validation(register_page, name, email, password, confirm, expected):
    if email == "NEW":
        email = unique_email()
    fill_register_form(register_page, name, email, password, confirm)
    register_page.wait_for("flash-error")
    assert expected in register_page.flash_messages("error")
    assert register_page.exists("register-form")              # still on the register page


@pytest.mark.parametrize("field", ["register-name-input", "register-email-input",
                                   "register-password-input", "register-confirm-input"],
                         ids=["TC_REG_08_empty_name", "TC_REG_09_empty_email",
                              "TC_REG_10_empty_password", "TC_REG_11_empty_confirm"])
def test_registration_empty_field(register_page, field):
    values = {"register-name-input": "Asha", "register-email-input": unique_email(),
              "register-password-input": "Passw0rd1", "register-confirm-input": "Passw0rd1"}
    values[field] = ""                                        # leave ONE field empty
    for testid, value in values.items():
        if value:
            register_page.type(testid, value)
    register_page.click("register-submit-btn")
    # The form has `novalidate`: the server checks it and shows one flash per problem.
    register_page.wait_for("flash-error")
    assert "This field is required." in register_page.flash_messages("error")
    assert register_page.exists("register-form")
```

Run it

```
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_02_register.py -v
```

You should see

```
11 passed in 41.36s
```

Explain in the viva

One reusable step (`fill_register_form`) is used by all 12 cases. The negative tests check two things: the exact error message **and** that we are still on the register form (registration did not happen). Testing boundary values matters: 5 characters is below the minimum of 8.

31 Step 31: Scenario 3: File upload and prediction

~15 min

Upload code and check the prediction report; then check every way an upload can be refused. File inputs are filled with `send_keys(absolute path)` : Selenium cannot use the Windows file dialog.

TC ID	Type	Input	Expected result
TC_UPL_01	Positive	data\samples\sample_project.zip + title	Report done; 8 files; high+medium+low = 8; riskiest file first
TC_UPL_02	Positive	a single .py file	Report with 1 file: pricing.py
TC_UPL_03	Negative	notes.txt	'notes.txt' is not a supported source file. ...
TC_UPL_04	Negative	an empty .py file	The uploaded file is empty.
TC_UPL_05	Empty	no file chosen	Please choose a file to upload.

TC ID	Type	Input	Expected result
TC_UPL_06	Negative	Git URL https://example.com/owner/repo	Host 'example.com' is not allowed. ...
TC_UPL_07	Empty	Git source, no URL	Please enter a Git repository URL.

tests\selenium\tests\test_exam_03_upload.py

```
"""EXAM SCENARIO 3 - FILE UPLOAD + PREDICTION (positive and negative)."""
from pathlib import Path

import pytest

from tests.selenium.pages import LoginPage, UploadPage

ROOT = Path(__file__).resolve().parents[3]
SAMPLE_ZIP = ROOT / "data" / "samples" / "sample_project.zip"
SINGLE_FILE = ROOT / "data" / "samples" / "sample_project" / "inventory" / "pricing.py"


@pytest.fixture
def upload_page(driver, base_url, timeout):
    LoginPage(driver, base_url, timeout).login_as("dev@test.com")   # precondition: logged in
    return UploadPage(driver, base_url, timeout).open()


def test_TC_UPL_01_upload_zip_shows_ranked_report(upload_page):
    report = upload_page.upload_file(SAMPLE_ZIP, title="Exam ZIP upload")
    assert report.is_done
    assert report.title == "Exam ZIP upload"
    assert "Analysis complete: 8 file(s) analysed." in report.flash_messages("success")
    summary = report.summary()
    assert summary["files"] == "8"
    assert int(summary["high"]) + int(summary["medium"]) + int(summary["low"]) == 8
    assert report.file_paths()[0] == "legacy/report_builder.py"     # riskiest file first


def test_TC_UPL_02_upload_single_python_file(upload_page):
    report = upload_page.upload_file(SINGLE_FILE)
    assert report.is_done
    assert report.summary()["files"] == "1"
    assert report.file_paths() == ["pricing.py"]


def test_TC_UPL_03_unsupported_file_type(upload_page, tmp_path):
    notes = tmp_path / "notes.txt"
    notes.write_text("this is not source code")
    upload_page.select_file(notes)
    message = upload_page.submit_expecting_error()
    assert message == "'notes.txt' is not a supported source file. Upload a source file or a .zip archive."
    assert upload_page.current_path == "/upload"


def test_TC_UPL_04_empty_file(upload_page, tmp_path):
    empty = tmp_path / "empty.py"
    empty.write_text("")
    upload_page.select_file(empty)
    assert upload_page.submit_expecting_error() == "The uploaded file is empty."


def test_TC_UPL_05_no_file_chosen(upload_page):
    assert upload_page.submit_expecting_error() == "Please choose a file to upload."


def test_TC_UPL_06_git_host_not_allowed(upload_page):
    upload_page.choose_git_source()
    upload_page.type("git-url-input", "https://example.com/owner/repo")
    message = upload_page.submit_expecting_error()
    assert message.startswith("Host 'example.com' is not allowed.")


def test_TC_UPL_07_git_url_empty(upload_page):
    upload_page.choose_git_source()
```

```
tests\selenium\tests\test_exam_03_upload.py
66      assert upload_page.submit_expecting_error() == "Please enter a Git repository URL."
```

Run it

```
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_03_upload.py -v
```

You should see

```
7 passed in 46.94s
```

Explain in the viva

The fixture `upload_page` is the precondition (logged in as Developer, on /upload). Test files for the negative cases are created on the fly in pytest's `tmp_path` folder, so no extra files are needed. The analysis takes a few seconds; `upload_file` waits for the sync marker `status-done` instead of sleeping.

32 Step 32: Scenario 4: Report page, sorting, dynamic content, new tab

~15 min

Verify the results on report #1, sort the table, open a hidden drill-down, check export links, open the report in a **second browser tab**, and check that another Developer cannot see it.

TC ID	What is checked	Expected result
TC_REP_01	Summary cards against the table	8 files; Medium/High/Low counts match the badges; charts drawn
TC_REP_02	Default order	Probabilities sorted high to low, all between 0 and 1
TC_REP_03	Click the File header twice	A to Z, then Z to A
TC_REP_04	Drill-down of inventory/pricing.py	Hidden first; after the click visible with 2 Low units
TC_REP_05	Export links	PDF, CSV and unit CSV links end with the right URL
TC_REP_06	Open the report in a new tab	2 windows; right title; close it and return to the first tab
TC_REP_07	Another Developer opens /reports/1	403 or 404: not allowed

```
tests\selenium\tests\test_exam_04_report.py
1  """EXAM SCENARIO 4 - REPORT PAGE: verify results, sort a table, dynamic drill-down,
2  export links, and open the report in a NEW TAB (window handling)."""
3  from selenium.webdriver.common.by import By
4  from selenium.webdriver.support import expected_conditions as EC
5
6  from tests.selenium.pages import LoginPage, ReportPage
7
8
9  def open_seed_report(driver, base_url, timeout):
10      LoginPage(driver, base_url, timeout).login_as("dev@test.com")
11      return ReportPage(driver, base_url, timeout).open_report(1)       # report #1 is created by the seed
12
13
14  def test_TC_REP_01_summary_matches_table(driver, base_url, timeout):
15      report = open_seed_report(driver, base_url, timeout)
16      assert report.title == "Sample project (seed)"
17      assert report.is_done
18      summary = report.summary()
19      paths = report.file_paths()
20      assert int(summary["files"]) == len(paths) == 8
21      risks = [report.risk_of(p) for p in paths]
22      assert risks.count("Medium") == int(summary["medium"])
23      assert risks.count("High") == int(summary["high"])
24      assert risks.count("Low") == int(summary["low"])
25      assert report.charts_rendered()
26
```

tests\selenium\tests\test_exam_04_report.py

```
27
28  def test_TC_REP_02_default_order_is_probability_descending(driver, base_url, timeout):
29      report = open_seed_report(driver, base_url, timeout)
30      probs = [report.probability_of(p) for p in report.file_paths()]
31      assert probs == sorted(probs, reverse=True)
32      assert all(0.0 <= p <= 1.0 for p in probs)
33
34
35  def test_TC_REP_03_sort_by_file_name(driver, base_url, timeout):
36      report = open_seed_report(driver, base_url, timeout)
37      before = report.file_paths()
38      assert report.sort_by("file") == "file:ascending"
39      assert report.file_paths() == sorted(before)
40      assert report.sort_by("file") == "file:descending"              # second click reverses
41      assert report.file_paths() == sorted(before, reverse=True)
42
43
44  def test_TC_REP_04_unit_drill_down(driver, base_url, timeout):
45      report = open_seed_report(driver, base_url, timeout)
46      table = "units-table-inventory-pricing-py"
47      assert not report.find(table).is_displayed()                    # hidden before the click
48      assert report.expand_units("inventory/pricing.py") == ["Low", "Low"]
49      assert report.find(table).is_displayed()
50
51
52  def test_TC_REP_05_export_links(driver, base_url, timeout):
53      report = open_seed_report(driver, base_url, timeout)
54      assert report.export_url("pdf").endswith("/reports/1/export.pdf")
55      assert report.export_url("csv").endswith("/reports/1/export.csv")
56      assert report.export_url("units-csv").endswith("/reports/1/export.csv?level=unit")
57
58
59  def test_TC_REP_06_open_report_in_new_tab(driver, base_url, timeout):
60      LoginPage(driver, base_url, timeout).login_as("dev@test.com")
61      original = driver.current_window_handle
62
63      driver.switch_to.new_window("tab")                              # open + switch to a new tab
64      driver.get(base_url + "/reports/1")
65      report = ReportPage(driver, base_url, timeout).wait_loaded()
66      assert len(driver.window_handles) == 2
67      assert report.title == "Sample project (seed)"
68
69      driver.close()                                                  # close the new tab
70      driver.switch_to.window(original)                               # back to the first tab
71      assert driver.find_element(By.CSS_SELECTOR, '[data-testid="dashboard-heading"]')
72
73
74  def test_TC_REP_07_report_of_another_user_is_forbidden(driver, base_url, timeout, new_user):
75      """A Developer may only see their OWN reports. Report #1 belongs to dev@test.com,
76      so a different (newly registered) Developer must be refused."""
77      page = LoginPage(driver, base_url, timeout).login_as(new_user, "Passw0rd1")
78      page.open("/reports/1")
79      assert page.error_code in ("403", "404")
```

Run it

```
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_04_report.py -v
```

You should see

```
7 passed in 39.08s
```

Explain in the viva

Window handling: every tab has a **handle**. `driver.switch_to.new_window("tab")` opens a tab and moves control there; `driver.close()` closes it; `driver.switch_to.window(original)` goes back. Without switching back, the next command would fail because Selenium would still point at the closed tab.

33 Step 33: Scenario 5: Dropdowns (filter, compare, admin role) and user management

~15 min

The app has four `<select>` dropdowns. Selenium's `Select` class handles them: by visible text, by value or by index.

TC ID	What is checked	Expected result
TC_DD_01	Reports filter: all options; choose Medium, then High	3 options; report #1 shown for Medium, not for High
TC_DD_02	QA compares report #1 with a new upload of the same ZIP	Files in both = 8; added 0; removed 0
TC_DD_03	Compare a report with itself	Please choose two different reports.
TC_DD_04	Admin changes a new user's role to QA	Saved; that user then logs in as qa
TC_DD_05	Admin deactivates a new user	Status inactive; login refused with the deactivated message

tests\selenium\tests\test_exam_05_dropdowns.py

```
"""EXAM SCENARIO 5 - DROPDOWNS (<select>): report filter, compare two reports,
and an admin changing a user's role."""
from selenium.webdriver.support.ui import Select

from tests.selenium.pages import LoginPage, UploadPage, path_key
from tests.selenium.tests.test_exam_03_upload import SAMPLE_ZIP


def test_TC_DD_01_reports_risk_filter(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).login_as("dev@test.com")
    page.open("/reports")
    dropdown = Select(page.find("risk-filter-select"))
    assert [o.text for o in dropdown.options] == [
        "All reports", "With high-risk files", "With medium-risk files"]

    dropdown.select_by_value("medium")              # the page reloads by itself (auto-submit)
    page.wait.until(lambda d: "risk=medium" in d.current_url)
    assert page.exists("report-list-row-1")         # the seed report has 1 medium-risk file
    assert Select(page.find("risk-filter-select")).first_selected_option.text == "With medium-risk files"

    Select(page.find("risk-filter-select")).select_by_visible_text("With high-risk files")
    page.wait.until(lambda d: "risk=high" in d.current_url)
    assert not page.exists("report-list-row-1")     # it has 0 high-risk files


def test_TC_DD_02_qa_compares_two_reports(driver, base_url, timeout):
    # Precondition: two completed reports. The developer uploads a second one.
    LoginPage(driver, base_url, timeout).login_as("dev@test.com")
    second = UploadPage(driver, base_url, timeout).open().upload_file(SAMPLE_ZIP, title="For comparison")
    new_id = second.report_id
    second.logout()

    page = LoginPage(driver, base_url, timeout).login_as("qa@test.com")
    page.open("/reports/compare")
    Select(page.find("compare-a-select")).select_by_value("1")
    Select(page.find("compare-b-select")).select_by_value(str(new_id))
    page.click("compare-submit-btn")
    page.wait_for("status-done")
    assert page.text("compare-common") == "8"       # same project -> all 8 files in common
    assert page.text("compare-added") == "0"
    assert page.text("compare-removed") == "0"


def test_TC_DD_03_compare_same_report_twice_is_rejected(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).login_as("qa@test.com")
    page.open("/reports/compare?a=1&b=1")
    assert "Please choose two different reports." in page.flash_messages("error")

```

tests\selenium\tests\test_exam_05_dropdowns.py

```
49
50  def test_TC_DD_04_admin_changes_role_with_dropdown(driver, base_url, timeout, new_user):
51      # Precondition: a fresh user (so the seeded demo accounts stay unchanged).
52      admin = LoginPage(driver, base_url, timeout).login_as("admin@test.com")   # returns a BasePage
53      admin.open("/admin/users")
54      admin.wait_for("users-table")
55      key = path_key(new_user)                      # "user_ab12@test.com" -> "user-ab12-test-com"
56      Select(admin.find(f"role-select-{key}")).select_by_value("qa")
57      admin.click(f"role-save-btn-{key}")
58      admin.wait_for("flash-success")
59      assert Select(admin.find(f"role-select-{key}")).first_selected_option.get_attribute("value") == "qa"
60
61      admin.logout()                                # the user now logs in with the QA role
62      user = LoginPage(driver, base_url, timeout).login_as(new_user, "Passw0rd1")
63      assert user.user_role == "qa"
64
65
66  def test_TC_DD_05_admin_deactivates_user_who_then_cannot_log_in(driver, base_url, timeout, new_user):
67      admin = LoginPage(driver, base_url, timeout).login_as("admin@test.com")
68      admin.open("/admin/users")
69      key = path_key(new_user)
70      admin.click(f"toggle-active-btn-{key}")
71      admin.wait_for("flash-success")
72      assert admin.text(f"user-status-{key}") == "inactive"
73      admin.logout()
74
75      login = LoginPage(driver, base_url, timeout).open()
76      login.login(new_user, "Passw0rd1")
77      assert login.error_message == "This account has been deactivated. Contact an administrator."
```

Run it

```
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_05_dropdowns.py -v
```

You should see

```
5 passed in 48.93s
```

Explain in the viva

The filter dropdown submits by itself when it changes, so the test waits for the URL to contain `risk=medium` before checking the table. Compare needs two finished reports, so the test first **creates its own test data** (the Developer uploads a second report). Tests that change users use the `new_user` fixture, so the demo accounts other tests rely on never change: tests stay independent.

34 Step 34: Scenario 6: Alerts, navigation, cookies, waits, scrolling, access control

~15 min

General browser skills from the exam sheet. **This app has no JavaScript alerts and no frames**, so the alert tests create an alert with `execute_script` to practise handling it. (For frames, see Appendix C and the selenium-exam-prep practice kit.)

TC ID	Skill	Expected result
TC_BR_01	Alert: read the text and accept (OK)	Text 'Session will expire soon'; alert closed
TC_BR_02	Confirm: dismiss (Cancel)	confirm() returned False
TC_BR_03	Title, Back, Forward, Refresh	Pages change as with the browser buttons
TC_BR_04	Delete cookies	Session cookie gone; /dashboard redirects to /login
TC_BR_05	Explicit wait that never comes true	TimeoutException after 2 s
TC_BR_06	Scroll to the table; screenshot of one element	PNG file created

TC ID	Skill	Expected result
TC_BR_07-09	Developer opens /admin/users; QA opens /upload; Admin opens /admin/users	403; 403; allowed

tests\selenium\tests\test_exam_06_browser_skills.py

```
"""EXAM SCENARIO 6 - GENERAL BROWSER SKILLS on this app: alerts, waits, navigation,
cookies/session, page title, keyboard, scrolling, element screenshot, access control."""
import pytest
from selenium.common.exceptions import TimeoutException
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

from tests.selenium.pages import LoginPage, ReportPage


def test_TC_BR_01_javascript_alert(driver, base_url, timeout):
    """The app itself has no alerts, so we create one with JavaScript to practise handling it."""
    LoginPage(driver, base_url, timeout).open()
    driver.execute_script("setTimeout(() => alert('Session will expire soon'), 500)")
    alert = WebDriverWait(driver, 5).until(EC.alert_is_present())
    assert alert.text == "Session will expire soon"
    alert.accept()


def test_TC_BR_02_javascript_confirm_dismiss(driver, base_url, timeout):
    LoginPage(driver, base_url, timeout).open()
    driver.execute_script("setTimeout(() => window.answer = confirm('Delete report?'), 300)")
    WebDriverWait(driver, 5).until(EC.alert_is_present()).dismiss()     # click Cancel
    assert driver.execute_script("return window.answer") is False


def test_TC_BR_03_page_titles_and_back_forward(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).login_as("dev@test.com")
    assert "Dashboard" in driver.title
    page.click("nav-reports")
    page.wait_for("reports-heading")
    driver.back()                                                   # browser Back button
    page.wait_for("dashboard-heading")
    driver.forward()                                                # browser Forward button
    page.wait_for("reports-heading")
    driver.refresh()
    assert page.exists("reports-heading")


def test_TC_BR_04_deleting_cookies_logs_user_out(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).login_as("dev@test.com")
    assert driver.get_cookie("session") is not None                 # Flask session cookie
    driver.delete_all_cookies()
    page.open("/dashboard")
    assert page.current_path.startswith("/login")                   # protected page -> login


def test_TC_BR_05_explicit_wait_timeout_demo(driver, base_url, timeout):
    """Waiting for something that never appears raises TimeoutException (a FAILED test in real use)."""
    LoginPage(driver, base_url, timeout).open()
    with pytest.raises(TimeoutException):
        WebDriverWait(driver, 2).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, '[data-testid="dashboard-heading"]')))


def test_TC_BR_06_scroll_and_element_screenshot(driver, base_url, timeout, tmp_path):
    LoginPage(driver, base_url, timeout).login_as("dev@test.com")
    report = ReportPage(driver, base_url, timeout).open_report(1)
    table = report.find("report-table")
    driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", table)
    shot = tmp_path / "table.png"
    assert table.screenshot(str(shot))                              # screenshot of ONE element
    assert shot.stat().st_size > 1000


@pytest.mark.parametrize("email, path, expected", [
```

tests\selenium\tests\test_exam_06_browser_skills.py

```
    ("dev@test.com", "/admin/users", "403"),
    ("qa@test.com", "/upload", "403"),
    ("admin@test.com", "/admin/users", None),
], ids=["TC_BR_07_dev_blocked_from_admin", "TC_BR_08_qa_blocked_from_upload", "TC_BR_09_admin_allowed"])
def test_role_based_access(driver, base_url, timeout, email, path, expected):
    page = LoginPage(driver, base_url, timeout).login_as(email)
    page.open(path)
    assert page.error_code == expected
```

Run it

```
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_06_browser_skills.py -v
```

You should see

```
9 passed in 44.80s
```

Explain in the viva

Alerts block the page: first wait with `EC.alert_is_present()`, then `accept()` (OK), `dismiss()` (Cancel), `.text` to read it, or `send_keys()` for a prompt. Access control is tested at the URL level, not only by hiding menu links: a user could type the address.

PART G

Write a new test yourself (what you do in the exam)

In the exam you will get a scenario you have not seen. Use this recipe every time.

35 Step 35: The 8-step recipe for any scenario

~5 min

#	Do this	Output
1	**Read** the scenario twice. Underline the user role, the page and the expected outcome.	Understanding
2	**Do it by hand** in the app (Step 11) with valid, invalid and empty input. Copy every message exactly.	Expected results
3	**Write test cases**: 1-2 positive, 3-5 negative, 1-2 empty. ID, steps, data, expected result.	Test-case table
4	**Find the locators**: F12 > Inspect > data-testid, or `docs\selenium-locators.md` .	List of data-testids
5	**Reuse page objects**: login_as, open, click, type, text, flash_messages, Select ...	Fewer lines
6	**Write the test file** `tests\selenium\tests\test_exam_XX_name.py` : precondition, actions, wait, asserts.	Code
7	**Run it** after each test you add. Read failures (Step 40), fix, run again.	All PASSED (or a defect)
8	**Evidence**: final run with the HTML report (Step 38) and fill in the test-case document.	Submission

36 Step 36: Worked example from scratch

~10 min

Scenario from the examiner: *"A Developer opens the upload page from the dashboard and finds a report through the reports list. A QA Engineer must not see the upload option."*

Recipe step	Result
2. By hand	Dashboard button "Analyze code" opens /upload with the heading "Analyze code". "My reports" lists "Sample project (seed)". As QA there is no Analyze button.
3. Test cases	TC_NEW_01 (positive) button opens upload; TC_NEW_02 (positive) list opens the report; TC_NEW_03 (negative) QA has no upload button or menu link.
4. Locators	`dashboard-analyze-btn` , `upload-form` , `upload-heading` , `nav-reports` , `reports-table` , `report-link-1` , `status-done` , `report-title`
5. Reuse	`LoginPage.login_as` , `click` , `wait_for` , `text` , `exists` , `nav_items`

tests\selenium\tests\test_exam_07_my_scenario.py

```
"""EXAM SCENARIO 7 - WORKED EXAMPLE written from scratch with the 8-step recipe (Part G).

Scenario given by the examiner:
  "A Developer opens the upload page from the dashboard and finds a report through the
   reports list. A QA Engineer must not see the upload option."
"""
from tests.selenium.pages import LoginPage

PASSWORD = "Test@1234"


def test_TC_NEW_01_dashboard_button_opens_upload_page(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).login_as("dev@test.com", PASSWORD)   # precondition
    page.click("dashboard-analyze-btn")                                               # action
    page.wait_for("upload-form")                                                      # wait (sync)
```

tests\selenium\tests\test_exam_07_my_scenario.py

```
    assert page.current_path == "/upload"                                    # verify
    assert page.text("upload-heading") == "Analyze code"


def test_TC_NEW_02_reports_list_opens_the_report(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).login_as("dev@test.com", PASSWORD)
    page.click("nav-reports")
    page.wait_for("reports-table")
    assert page.exists("report-list-row-1")
    page.click("report-link-1")
    page.wait_for("status-done")
    assert page.text("report-title") == "Sample project (seed)"


def test_TC_NEW_03_qa_does_not_see_upload(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).login_as("qa@test.com", PASSWORD)
    assert not page.exists("dashboard-analyze-btn")                          # negative check
    assert "nav-upload" not in page.nav_items()
```

Step 7: run it

```
.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_07_my_scenario.py -v
```

You should see

```
test_TC_NEW_01_dashboard_button_opens_upload_page PASSED
test_TC_NEW_02_reports_list_opens_the_report PASSED
test_TC_NEW_03_qa_does_not_see_upload PASSED
3 passed in 31.88s
```

37 Step 37: A blank template to start from

~2 min

Create a new file in `tests\selenium\tests` (Notepad: File > Save As, "All files", name it `test_exam_08_scenario.py`) and start from this skeleton. Replace every TODO.

Template

```
"""EXAM SCENARIO: TODO one-line description of the scenario."""
import pytest
from selenium.webdriver.support.ui import Select

from tests.selenium.pages import BasePage, LoginPage, UploadPage, ReportPage


def test_TC01_TODO_positive(driver, base_url, timeout):
    page = LoginPage(driver, base_url, timeout).login_as("dev@test.com")   # precondition
    page.open("/TODO")                                                      # go to the page
    page.type("TODO-input", "TODO valid value")                             # actions
    page.click("TODO-btn")
    page.wait_for("TODO-result-or-flash-success")                           # wait for the result
    assert page.text("TODO-result") == "TODO exact expected text"           # verify


@pytest.mark.parametrize("value, expected_error", [
    ("TODO invalid", "TODO exact error"),
    ("", "TODO exact error for empty"),
], ids=["TC02_TODO_invalid", "TC03_TODO_empty"])
def test_TODO_negative(driver, base_url, timeout, value, expected_error):
    page = LoginPage(driver, base_url, timeout).login_as("dev@test.com")
    page.open("/TODO")
    if value:
        page.type("TODO-input", value)
    page.click("TODO-btn")
    page.wait_for("flash-error")
    assert expected_error in page.flash_messages("error")
    assert not page.exists("TODO-success-element")                          # and NO success
```

If something goes wrong

- **IndentationError**: Python uses the spaces at the start of lines. Use 4 spaces per level and never mix tabs and spaces.
- **The test is not found** ("collected 0 items"): the file name must start with `test_` and the function name with `test_`.
- **TimeoutException on page.open()** after `LoginPage(...).open()`: a LoginPage object always waits for the login form. Use the object returned by `login_as()`, or a `BasePage`, to open other pages.

PART H

Results, evidence and submission

The exam sheet asks for test scripts, test data, execution results and screenshots, with failed tests clearly documented.

38 Step 38: The final run (results + screenshots in one go)

~5 min

Type these, pressing Enter after each line

```
Remove-Item -Recurse -Force tests\selenium\artifacts\evidence
.\.venv\Scripts\python.exe -m pytest tests\selenium -v --html=reports\selenium_report.html --self-contained-html
start reports\selenium_report.html
```

To submit only your exam scenario, put its file name instead of `tests\selenium`. Also take a screenshot of the PowerShell window with the summary line (Windows key + Shift + S).

39 Step 39: Fill in the test-case document

~10 min

Make a table in Word or Excel, one row per test case, with these columns. Fill in **Actual result** and **Status** from the HTML report after the final run.

Column	What to write	Example (TC_LOGIN_04)
TC ID	Unique ID, the same as in the test name	TC_LOGIN_04
Scenario / module	Which feature	Login
Type	Positive / Negative / Empty / Boundary	Negative
Pre-condition	What must be true before the steps	On /login, logged out
Steps	Numbered actions	1. Type email 2. Type password 3. Click Log in
Test data	The exact values	dev@test.com / Wrong@1234
Expected result	Exact message or state	Error 'Invalid email or password.'; not logged in
Actual result	What happened (from the report)	As expected
Status	PASS / FAIL	PASS
Evidence	Screenshot file name	PASS_test_login_data_driven_TC_LOGIN_04_wrong_password__....png

40 Step 40: Read a failure and document it

~10 min

When a test FAILS, pytest prints the file, the line and the reason. Read it from the bottom up:

```
Example: a real failure from an earlier version of the file, met while writing this guide
    assert expected in register_page.flash_messages("error")
E   AssertionError: assert 'Field must be at least 8 characters long.' in
E                   ['Field must be between 8 and 128 characters long.']
tests\selenium\tests\test_exam_02_register.py:56: AssertionError
```

1. **Which test and line?** test_exam_02_register.py, line 56 (of that earlier version).
2. **Expected vs actual?** The left side is what the test expected; the list is what the page showed.
3. **Look at the evidence**: the FAIL_ screenshot (and on failures the saved HTML in `tests\selenium\artifacts`).

4. **Decide**: is the **test** wrong (wrong locator, wrong expected text, missing wait) or is the **app** wrong (a defect)? Check by doing the same steps by hand.

Three real failures met while writing this guide, and how they were handled:

Failure	Cause	Kind	Action
Expected 'at least 8 characters', page said 'between 8 and 128 characters'	Expected text guessed instead of copied	Script/data issue	Copied the real message; test passes
Empty-field login test passed alone but failed in a full run with StaleElementReferenceExceptio n	Assumed the browser blocks empty fields. The form has *novalidate*, so it was sent and the page reloaded while the test was reading it	Script issue (wrong assumption)	Asserted the server's message instead; stable in 3 runs
TimeoutException when opening /admin/users	Used a LoginPage object, whose open() waits for the login form	Script issue	Used the page returned by login_as() (a BasePage)

If it is an **application defect**, keep the test (do not delete or weaken it), and document it:

Field	Example
Defect ID / title	D01: Upload accepts a 0-byte .zip file without an error
Test case	TC_UPL_08 (test_exam_03_upload.py)
Steps to reproduce	1. Log in as dev@test.com 2. Open Analyze code 3. Choose empty.zip 4. Click Analyze
Expected / Actual	Error message / (what really happened)
Severity	Medium (wrong input accepted)
Evidence	FAIL_test_TC_UPL_08_....png, report.html

(The defect above is an *example of the format*, not a known bug. All 70 tests in this guide pass.)

41 Step 41: Pack the submission

~3 min

Type this (replace RegNo_Name with yours)

```
Compress-Archive -Path tests\selenium, reports -DestinationPath C:\SDP\RegNo_Name_Selenium.zip -Force
```

You should see

The file `C:\SDP\RegNo_Name_Selenium.zip` (about 10 MB) contains the test scripts, page objects, test data, the evidence screenshots and the HTML report. Add your test-case document and submit as instructed.

PART I

Exam day

42 Step 42: The evening before and the morning of the exam

~20 min

When	Check	Done
Evening	Laptop charged; charger packed	[ ]
Evening	Step 19 passes (70 passed); screenshot of the summary taken	[ ]
Evening	Step 18 first script passes with the app running	[ ]
Evening	You can explain Parts C and E in your own words; read Part J aloud	[ ]
Morning	Connected to the exam Wi-Fi (the first run of the day may download the driver)	[ ]
Morning	PowerShell open in `C:\SDP\defect-predictor`; this guide and an editor open	[ ]
Morning	Quick check: `.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_login.py` shows 5 passed	[ ]

43 Step 43: Time plan for 1 hour 45 minutes

~1 min

Time	Minutes	Do
9:45-9:55	10	Read the scenario. Do it by hand: valid, invalid, empty. Write down every message exactly.
9:55-10:05	10	Write the test cases (IDs, data, expected).
10:05-10:15	10	Find the locators (F12, docs\selenium-locators.md).
10:15-10:50	35	Write the test file from the template (Step 37). Run after every test. Fix.
10:50-11:05	15	Add data-driven cases (parametrize or CSV), extra negative cases, dropdown/alert/tab if the scenario has them.
11:05-11:15	10	Final run with the HTML report (Step 38). Fill in actual results and status.
11:15-11:30	15	Pack and submit (Step 41). Demo and viva: run once with SDP_HEADLESS=0 so the examiner sees the browser.

Exam tip

Golden rules: copy expected messages exactly; run early and often; every test ends with an assert; negative tests check the error **and** that nothing succeeded; never use time.sleep; never delete a test that found a real defect: document it.

PART J

Viva questions and answers

Q. What is Selenium? Which part did you use?

An open-source tool for automating web browsers. I used **Selenium WebDriver** (Python), which drives a real Chrome through chromedriver. Other parts: Selenium IDE (record and playback) and Selenium Grid (run tests on many machines in parallel).

Q. How does WebDriver control Chrome?

My Python code calls the Selenium library, which sends W3C WebDriver commands over HTTP to chromedriver, which controls Chrome. Selenium Manager downloads the matching chromedriver automatically.

Q. What did you test?

The Software Defect Prediction System, a Flask web app: login, registration, upload and prediction, report page, reports filter, compare, admin user management and role-based access. 70 Selenium tests, positive, negative and empty-input cases.

Q. Which locators did you use and why?

data-testid attributes through CSS selectors, e.g. [data-testid="login-email-input"]. They are unique, made for testing and do not change with styling. Preference: data-testid, then ID, NAME, CSS, XPath last.

Q. What is the Page Object Model? Why use it?

One class per page with its locators and actions (LoginPage, UploadPage, ReportPage, all based on BasePage). Tests become short and readable, steps are reused, and a UI change is fixed in one place.

Q. What is a fixture? Which ones do you have?

Setup/teardown code pytest provides to tests. base_url starts an isolated, seeded copy of the app once per run; _browser starts Chrome once; driver gives each test a clean, logged-out browser; new_user registers a fresh account for tests that change users.

Q. How do you make tests independent and repeatable?

Each run uses a fresh temporary database with the same seed data; each test starts with cookies cleared on /login; tests that change data create their own (unique emails, new_user, their own uploads).

Q. Implicit vs explicit wait? Why not sleep?

Implicit: a global timeout for finding elements. Explicit: WebDriverWait(...).until(condition) for a specific condition. I use explicit waits and sync markers like status-done. sleep always wastes time and still fails if the page is slower.

Q. How do you handle a dropdown?

With the Select class: select_by_visible_text, select_by_value, select_by_index; first_selected_option to verify. Used for the risk filter, compare and role dropdowns.

Q. How do you upload a file?

send_keys with the absolute path on the input type=file element. Selenium cannot operate the operating system's file dialog.

Q. How do you handle alerts?

Wait with EC.alert_is_present(), then alert.text, accept(), dismiss() or send_keys(). This app has none, so my tests create one with execute_script to practise.

Q. How do you handle multiple windows or tabs?

Each window has a handle. I save current_window_handle, open a tab with switch_to.new_window('tab'), work there, close it and switch_to.window back.

Q. And frames?

switch_to.frame(id, name, index or element) before using elements inside an iframe, switch_to.default_content() afterwards. This app has no frames.

Q. What is data-driven testing here?

The login test reads login_data.csv; parametrize runs the same test once per row with a readable ID. Registration, upload and access tests are parametrized too.

Q. Positive vs negative vs boundary testing?

Positive: valid data, expect success. Negative: invalid data, expect the exact error and no success. Boundary: values at the limit, e.g. a 5-character password when 8 is the minimum.

Q. How do you verify results?

Assertions on URL path, visible text, flash messages, attributes (data-role, data-risk), element state (displayed, selected) and counts that must match (summary vs table).

Q. Where are your results and evidence?

pytest-html report (reports\selenium_report.html) with an embedded screenshot per test, PASS_/FAIL_ screenshots in artifacts\evidence, and for failures also the page HTML.

Q. What did you do when a test failed?

Read the assertion and screenshot, reproduced it by hand, decided whether it was a script issue or an app defect. Example: an empty-field test failed because the form has novalidate, so I asserted the server's message instead.

Q. What is StaleElementReferenceException?

The page changed after an element was found, so the reference is old. Find the element again. LoginPage uses staleness on purpose to detect that the page has reloaded.

Q. What is headless mode?

Chrome runs without a window: faster, and used on servers and in CI. SDP_HEADLESS=0 shows the window.

Q. Why test access control by URL?

Hiding a menu link is not security; a user can type the URL. The tests open forbidden URLs directly and expect 403.

Q. Limitations of Selenium?

Web only (mobile needs Appium); cannot handle OS dialogs or CAPTCHAs; no built-in reports (pytest-html adds them); can be flaky without proper waits; slower than unit tests.

Q. Verification vs validation?

Verification: are we building the product right (meets its specification)? Validation: are we building the right product (meets user needs)? These automated tests are verification.

PART K

Troubleshooting

Problem	Cause and fix
'py' is not recognized	Python not installed with the launcher / PATH. Step 2, then reopen PowerShell.
No module named selenium / pytest	Wrong Python. Always use `.\.venv\Scripts\python.exe`.
SKIPPED: Chrome / chromedriver not available	Install or update Chrome (Step 3); be online for the first run.
SessionNotCreatedException: version mismatch	Chrome updated during the day. Restart Chrome, run again (Selenium Manager fetches the new driver).
NoSuchElementException	Wrong data-testid, or the page is not loaded. Check with $$() in DevTools, wait first.
TimeoutException	The awaited element never appeared: wrong locator, wrong page, or slow laptop ($env:SDP_TIMEOUT="60"). Check the screenshot.
ElementClickInterceptedException	Something covers the element (pop-up, overlay). Close it or scroll to the element; the upload radio buttons are clicked with JavaScript for this reason.
StaleElementReferenceException	The page reloaded. Find the element again after the action.
AssertionError 'X' == 'Y'	The test ran but the text differs. Compare character by character; copy the real message.
Typing ... did not stick	A browser pop-up took keyboard focus. The project's Chrome options switch these off; close other Chrome pop-ups and run again.
Address already in use (run.py)	The app is already running in another window. Use that one or $env:PORT="5001".
IndentationError / SyntaxError	Spaces at the start of a line are wrong, or a quote/bracket is missing.
collected 0 items	File or function name does not start with test_, or you are in the wrong folder (Step 6).
Tests pass alone but fail together	They share data. Use unique data (uuid, new_user) and never depend on the order of tests.

APPENDIX A

Command card

Every command is on one line, so it is safe to copy and paste. Run them inside `C:\SDP\defect-predictor` (Step 6).

Task	Command
Go to the project	`cd C:\SDP\defect-predictor`
Start the app (stop: Ctrl+C)	`.\.venv\Scripts\python.exe run.py`
Reset demo data	`.\.venv\Scripts\python.exe scripts\reset_db.py`
First script (app running)	`.\.venv\Scripts\python.exe tests\selenium\selenium_first_script.py`
All Selenium tests	`.\.venv\Scripts\python.exe -m pytest tests\selenium -v`
One file	`.\.venv\Scripts\python.exe -m pytest tests\selenium\tests\test_exam_01_login.py -v`
By keyword	`.\.venv\Scripts\python.exe -m pytest tests\selenium -k upload -v`
Show the browser	`$env:SDP_HEADLESS="0"`
Hide it again	`Remove-Item Env:SDP_HEADLESS`
Longer waits	`$env:SDP_TIMEOUT="60"`
HTML report	`.\.venv\Scripts\python.exe -m pytest tests\selenium --html=reports\selenium_report.html --self-contained-html`
Open the report	`start reports\selenium_report.html`
Clear evidence	`Remove-Item -Recurse -Force tests\selenium\artifacts\evidence`
Whole project suite	`.\.venv\Scripts\python.exe -m pytest -q`
Pack submission	`Compress-Archive -Path tests\selenium, reports -DestinationPath C:\SDP\Sub.zip -Force`

APPENDIX B

Locator reference (most used data-testids)

Full list: `docs\selenium-locators.md` . In code: `page.find("login-email-input")` or `driver.find_element(By.CSS_SELECTOR, '[data-testid="login-email-input"]')` .

Page	data-testids
Every page	main-nav, nav-dashboard, nav-upload, nav-reports, nav-compare, nav-admin-users, nav-user-role (attribute data-role), logout-btn, flash-success, flash-error, flash-info, loading-indicator, error-code (403/404)
/login	login-form, login-email-input, login-password-input, login-remember-checkbox, login-submit-btn, login-register-link
/register	register-form, register-name-input, register-email-input, register-password-input, register-confirm-input, register-submit-btn
/dashboard	dashboard-heading, dashboard-analyze-btn, stat-reports, stat-files, stat-high, recent-reports-table, recent-report-link-<id>
/upload	upload-form, source-file-radio, source-git-radio, upload-file-input, git-url-input, report-title-input, analyze-btn
/reports	risk-filter-select, reports-table, report-list-row-<id>, report-link-<id>, report-status-<id>
/reports/<id>	report-id, report-title, status-done, status-failed, summary-files/high/medium/low/max-prob/units, risk-chart, top-files-chart, report-table, sort-file, sort-probability, risk-badge-<path-key>, prob-<path-key>, units-toggle-<path-key>, export-pdf-btn, export-csv-btn, export-units-csv-btn
/reports/compare	compare-a-select, compare-b-select, compare-submit-btn, compare-common, compare-added, compare-removed, compare-table
/admin/users	users-table, role-select-<email-key>, role-save-btn-<email-key>, user-status-<email-key>, toggle-active-btn-<email-key>

path-key / email-key: lower-case, every run of other characters becomes a dash: `legacy/report_builder.py` becomes `legacy-report-builder-py` ; `qa@test.com` becomes `qa-test-com` . The helper `path_key()` does it for you.

APPENDIX C

Selenium cheat sheet (Python)

```
Everything in one place
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC

driver.get(url); driver.back(); driver.forward(); driver.refresh()
driver.title; driver.current_url; driver.page_source
el = driver.find_element(By.CSS_SELECTOR, '[data-testid="x"]')    # one (or NoSuchElementException)
els = driver.find_elements(By.CSS_SELECTOR, ".row")               # list (empty if none)
el.click(); el.clear(); el.send_keys("text", Keys.ENTER)
el.text; el.get_attribute("value"); el.is_displayed(); el.is_enabled(); el.is_selected()

wait = WebDriverWait(driver, 10)
wait.until(EC.visibility_of_element_located((By.ID, "msg")))
wait.until(EC.element_to_be_clickable((By.ID, "btn"))).click()
wait.until(EC.text_to_be_present_in_element((By.ID, "msg"), "Saved"))
wait.until(EC.url_contains("/dashboard"))

s = Select(driver.find_element(By.ID, "risk-filter-select"))       # dropdowns
s.select_by_visible_text("With medium-risk files"); s.select_by_value("high"); s.select_by_index(0)
s.first_selected_option.text; [o.text for o in s.options]

driver.find_element(By.ID, "upload-file-input").send_keys(r"C:\full\path\file.zip")   # upload

alert = wait.until(EC.alert_is_present())                          # alerts
alert.text; alert.accept(); alert.dismiss(); alert.send_keys("x")

main = driver.current_window_handle                                # windows / tabs
driver.switch_to.new_window("tab"); driver.switch_to.window(main); driver.window_handles; driver.close()

driver.switch_to.frame("frame-id")                                 # frames (iframes)
driver.switch_to.parent_frame(); driver.switch_to.default_content()

ActionChains(driver).move_to_element(el).perform()                 # hover
ActionChains(driver).double_click(el).perform(); ActionChains(driver).context_click(el).perform()
driver.execute_script("arguments[0].scrollIntoView();", el)        # JavaScript
driver.get_cookies(); driver.delete_all_cookies()
driver.save_screenshot("page.png"); el.screenshot("element.png")
```

> **Tip**
> Practise frames, hover and double-click (which this app does not have) with the offline practice site in the separate kit: https://github.com/Debdeeptax/selenium-exam-prep (practice_site\advanced.html).